Universidad Distrital Francisco José de Caldas
Ingeniería Electrónica · Análisis y Diseño de Microprocesadores

TALLER 5

Microprocesador de propósito específico: determinante por el algoritmo de Bareiss con salida BCD (algoritmo XS3)

Diseño RTL: unidad de control (FSM) y camino de datos
Diagrama ASM, señales de control y de estado, y código VHDL

Herramienta: Xilinx ISE 14.7 (XST, ISim) · Tarjeta: Digilent Nexys A7-100T (XC7A100T-1CSG324C)

Nota para el equipo. Este documento explica un diseño de referencia. El enunciado advierte que los diseños parecidos entre equipos no se califican, así que revisen cada estado y cada señal, entiendan por qué está ahí y adáptenlo antes de entregarlo. Las conclusiones del informe son responsabilidad del equipo.

Contenido

1. Resumen del diseño **3**
- Restricciones de diseño RTL . . . 3
- Requisitos del enunciado . . . 3
- Archivos del repositorio . . . 4

2. Repaso del algoritmo de Bareiss **4**
- Ejemplo 3×3 (página 1 de la ROM) . . . 4
- Pivote cero . . . 4

3. Algoritmo XS3 dentro del ASM **4**
- Traza del XS3 para 362 (determinante de la página 3) . . . 5

4. Caja negra, cajas grises y jerarquía **7**

5. Pseudocódigo con el hardware disponible **8**

6. Diagrama ASM **10**
- Convenciones . . . 10
- Vista general . . . 10
- Hoja 1: espera y carga de la matriz (S0 a S3) . . . 11
- Hoja 2: pivote, búsqueda de fila e intercambio (S4 a S10) . . . 12
- Hoja 3: productos y numerador (S11 a S16) . . . 13
- Hoja 4: división por restas sucesivas y avance (S17 a S19) . . . 14
- Hoja 5: determinante y valor a mostrar (S20 y S21) . . . 15
- Hoja 6: algoritmo XS3 paso a paso y fin (S22 a S26) . . . 16

7. Unidad de control **17**
- Codificación de estados . . . 17
- Palabra de control por estado . . . 17

8. Señales del procesador **20**
- 8.1 Señales de estado (CD → UC) . . . 20
- 8.2 Señales de control (UC → CD) . . . 20
- 8.3 Registros y tamaños . . . 21
- Por qué estos anchos . . . 21

9. Camino de datos **22**
- Cómo está escrito cada elemento . . . 22

10. Tiempo de ejecución **22**

11. Implementación en ISE 14.7 y la Nexys A7 **23**
- 11.1 Proyecto . . . 23
- 11.2 Operación en la tarjeta . . . 23
- 11.3 Matrices de prueba en la ROM . . . 23
- 11.4 Pines (src/nexys_a7.ucf) . . . 24

12. Simulación **24**

13. Limitaciones y verificación pendiente **25**

Anexo A. Código VHDL completo (src/det_bareiss.vhd) **26**

1. Resumen del diseño

El procesador lee de una ROM una matriz simétrica de 2×2 a 5×5, la copia a una RAM de trabajo y calcula su determinante con el algoritmo de Bareiss. Usa un multiplicador, un sumador/restador y una división por restas sucesivas. Guarda los pivotes en un banco de registros y convierte a BCD el valor que se va a mostrar con el algoritmo XS3 (sumar 3 y desplazar).

El diseño está dividido como en las notas de clase: una **unidad de control** (FSM de 27 estados, S0 a S26, que implementa el diagrama ASM) y un **camino de datos** (registros, multiplexores, memorias y unidades funcionales). La unidad de control le envía 54 señales de control (66 bits) y recibe 14 señales de estado. Todas las entidades están en un solo archivo, src/det_bareiss.vhd.

Restricciones de diseño RTL

Restricción	Cómo se cumple
Sin variables	No hay ninguna `variable` en el diseño. Solo el testbench, que no se sintetiza, las usa.
Solo std_logic y std_logic_vector	Todas las señales, puertos y constantes son std_logic o std_logic_vector. El estado de la FSM es un std_logic_vector(4 downto 0) con constantes S0 a S26. No hay integer, signed, unsigned, enumerados ni genéricos. **Única excepción:** la RAM se describe como `array of std_logic_vector`, que es la forma en que XST infiere una memoria (igual que el modelo de RAM del aula).
Construcciones sintetizables	Sin `wait`, `after`, archivos ni lazos `for`/`while` en el diseño. La síntesis genérica con GHDL termina sin errores y sin latches.
RTL claro: registros, lógica combinacional, FSM	Cada registro tiene sus señales de carga (ld_*, clr_*, inc_*). Cada multiplexor tiene su selección (sel_*) y está escrito con `with … select`. Las unidades funcionales son entidades combinacionales.
Separación FSM / camino de datos	Entidades `unidad_control` y `camino_datos`, unidas por la entidad estructural `procesador`.
Nada de descripciones tipo software	Las repeticiones del algoritmo (filas, columnas, pasos k, restas de la división, dígitos y bits del XS3) las hace la FSM, un paso por ciclo de reloj, con contadores del camino de datos.
Compatible con el esquemático RTL	Jerarquía estructural (figura 3). En ISE, «View RTL Schematic» muestra cada bloque como un símbolo.

Requisitos del enunciado

Requisito del taller	Cómo lo cumple el diseño
ASM con señal de inicio y de final	La FSM arranca con `inicio` (BTNC) y pone `final = '1'` solo en S26.
Matriz en ROM: 00h = n, luego los a(i,j) como en la Figura 1	ROM de 256 × 6 bits escrita con `with dir select`, 8 páginas de 32 palabras. En cada página, 00h = n y los elementos van por columnas (a11, a21, ...). `dira` de 5 bits; datos en complemento a 2 (−32 a 31).
Multiplicador, sumador/restador y división por restas sucesivas	Entidades `multiplicador` (con signo, 25 × 25) y `sumador_restador` (50 bits). La división son los registros R, Q y |p| del camino de datos más el lazo del estado S17, que usa el restador.
Memoria RAM para las iteraciones	Entidad `ram_trabajo` de 64 × 32 bits; dirección fila & columna.
Pivotes en un banco de registros, visibles con los switches	Registros b1 a b4. SW2..SW0 = 0 muestra el determinante; SW2..SW0 = k muestra el pivote k.
Determinante en los displays	Entidad `display_8dig`: AN7 = signo, AN6..AN0 = 7 dígitos; LD1 = desborde.
XS3 evidenciado paso a paso en el ASM	Un estado por paso del documento XS3, cada uno con su señal de control: S22 (clr_bcd), S23 (ins_bcd), S24 (aj_bcd) y S25 (sh_bcd).

Archivos del repositorio

Archivo	Contenido
src/det_bareiss.vhd	Diseño completo: 10 entidades (ROM, RAM, multiplicador, sumador/restador, camino de datos, unidad de control, procesador, sincronizador, display y tope).
src/nexys_a7.ucf	Pines y restricción de reloj (100 MHz) para ISE.
sim/tb_procesador.vhd	Testbench de la unidad de control + camino de datos con las 8 matrices y sus pivotes.
sim/tb_det_bareiss.vhd	Testbench del tope: lee el display de 7 segmentos (an, seg).
sim/modelo_referencia.py	Modelo en Python: det y pivotes esperados, ciclos y líneas de la ROM.
sim/resultado_ghdl.txt	Salida de las simulaciones.
doc/figuras/*.png, doc/fuentes/*.py	Diagramas y scripts que generan las figuras y este PDF.

2. Repaso del algoritmo de Bareiss

Bareiss es una eliminación gaussiana sin fracciones. En el paso k, cada elemento de la submatriz que está debajo y a la derecha del pivote se actualiza con la fórmula (1) del enunciado:

```
a(i,j) = ( a(k,k)·a(i,j) − a(i,k)·a(k,j) ) / p        para k < i ≤ n,  k < j ≤ n
p = pivote del paso anterior (p = 1 en el primer paso)
```

Después del paso k, el pivote actual a(k,k) pasa a ser p. Tras n − 1 pasos, el determinante queda en a(n,n). La división siempre es exacta: por la identidad de Sylvester, después del paso k cada a(i,j) es el determinante de una submatriz de (k+1)×(k+1) de A, que es un entero. En la simulación el residuo R siempre termina en 0.

Durante el paso k solo cambian los elementos con i > k y j > k. La fila k y la columna k no se modifican, por eso el cálculo se hace sobre la misma RAM, sin una segunda copia de la matriz.

Ejemplo 3×3 (página 1 de la ROM)

```
A = | 2 1 3 |      k = 1, p = 1:  a22 = (2·4 − 1·1)/1 = 7    a23 = (2·2 − 1·3)/1 = 1
    | 1 4 2 |                      a32 = (2·2 − 3·1)/1 = 1    a33 = (2·5 − 3·3)/1 = 1
    | 3 2 5 |      k = 2, p = 2:  a33 = (7·1 − 1·1)/2 = 3   →   det = 3
Pivotes guardados en el banco: pivote(1) = 2, pivote(2) = 7
```

Pivote cero

Si a(k,k) = 0, la FSM busca la primera fila i > k con a(i,k) ≠ 0 (estado S5), intercambia las filas k e i en las columnas k a n (S6 a S9) e invierte el bit `signo`. Al final, det = −a(n,n) si hubo un número impar de intercambios. Si no hay ninguna fila con a(i,k) ≠ 0, det = 0. Las columnas menores que k no se intercambian porque el algoritmo no las vuelve a leer.

3. Algoritmo XS3 dentro del ASM

El documento XS3 describe 5 pasos. En el ASM cada paso es un estado propio con su señal de control, de modo que el diagrama muestra la conversión completa y no una caja que diga «convertir a BCD».

Paso del documento XS3	Estado	Señal de control	Operación en el camino de datos
1. Inicializar el registro decimal en ceros	S22	clr_bcd, clr_cont	BCD ← 0 (40 bits); cont ← 0
2. Colocar el MSB del registro binario en el LSB del registro decimal	S23	ins_bcd	BCD ← BCD(39..1) & bin(31)

Paso del documento XS3	Estado	Señal de control	Operación en el camino de datos
3. Revisar cada dígito: si es ≥ 5 (> 0100), sumarle 3 (0011)	S24	aj_bcd (si dig_gt4), inc_d	Un dígito por ciclo (d = 0 a 9). El multiplexor de dígitos entrega BCD(d); el sumador calcula BCD(d) + 3; solo ese dígito se reemplaza.
4. Correr a la izquierda y descartar el bit que sale; repetir desde el paso 2	S25	sh_bcd, ld_bin (sel_bin = 10), inc_cont	BCD ← BCD(38..0) & 0; bin ← bin(30..0) & 0; cont ← cont + 1
5. Realizar únicamente el paso 2	S23 con cont_31	ins_bcd	En la pasada 32 por S23 entra el último bit y la FSM va a S26 sin pasar por S24 ni S25.

Por qué se suma 3: si un dígito vale 5 o más, al correrlo a la izquierda quedaría en 10 o más, que no es un dígito BCD válido. Sumar 3 antes del corrimiento equivale a sumar 6 después, que es la corrección BCD, y hace que el exceso pase como acarreo al dígito siguiente.

Tamaños: el registro binario `bin` tiene 32 bits, así que el registro decimal necesita 10 dígitos ($2^{32} - 1 = 4\,294\,967\,295$) y el lazo da 32 pasadas por el paso 2 (cont = 0 a 31). El signo no entra al XS3: en S21 se carga |valor| en `bin` y el signo en `neg`, que luego enciende el segmento g de AN7.

Traza del XS3 para 362 (determinante de la página 3)

$362 = 1\ 0110\ 1010_2$ tiene 9 bits significativos. En las pasadas cont = 0 a 22 solo entran ceros y el registro decimal sigue en 0, por eso la tabla empieza en cont = 23. La columna «bin(31..23)» muestra los 9 bits de arriba del registro binario; el bit de la izquierda, bin(31), es el que copia el paso 2. El registro decimal se muestra con sus tres dígitos bajos: centenas, decenas y unidades.

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
23	S23 (2)	0000 0000 0001	101101010	
	S24 (3)	0000 0000 0001	101101010	ningún dígito > 4
	S25 (4)	0000 0000 0010	011010100	corrimiento a la izquierda
24	S23 (2)	0000 0000 0010	011010100	
	S24 (3)	0000 0000 0010	011010100	ningún dígito > 4
	S25 (4)	0000 0000 0100	110101000	corrimiento a la izquierda
25	S23 (2)	0000 0000 0101	110101000	
	S24 (3)	0000 0000 1000	110101000	BCD(0) = 0101 > 0100: +3 = 1000
	S25 (4)	0000 0001 0000	101010000	corrimiento a la izquierda
26	S23 (2)	0000 0001 0001	101010000	
	S24 (3)	0000 0001 0001	101010000	ningún dígito > 4
	S25 (4)	0000 0010 0010	010100000	corrimiento a la izquierda
27	S23 (2)	0000 0010 0010	010100000	
	S24 (3)	0000 0010 0010	010100000	ningún dígito > 4
	S25 (4)	0000 0100 0100	101000000	corrimiento a la izquierda
28	S23 (2)	0000 0100 0101	101000000	
	S24 (3)	0000 0100 1000	101000000	BCD(0) = 0101 > 0100: +3 = 1000
	S25 (4)	0000 1001 0000	010000000	corrimiento a la izquierda
29	S23 (2)	0000 1001 0000	010000000	
	S24 (3)	0000 1100 0000	010000000	BCD(1) = 1001 > 0100: +3 = 1100
	S25 (4)	0001 1000 0000	100000000	corrimiento a la izquierda

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
30	S23 (2)	0001 1000 0001	100000000	
	S24 (3)	0001 1011 0001	100000000	BCD(1) = 1000 > 0100: +3 = 1011
	S25 (4)	0011 0110 0010	000000000	corrimiento a la izquierda
31	S23 (2)	0011 0110 0010	000000000	último PASO 2 = PASO 5, va a S26

Resultado: BCD = 0011 0110 0010 = 3 6 2. La simulación muestra « 0000362» en el display para esta matriz.

4. Caja negra, cajas grises y jerarquía

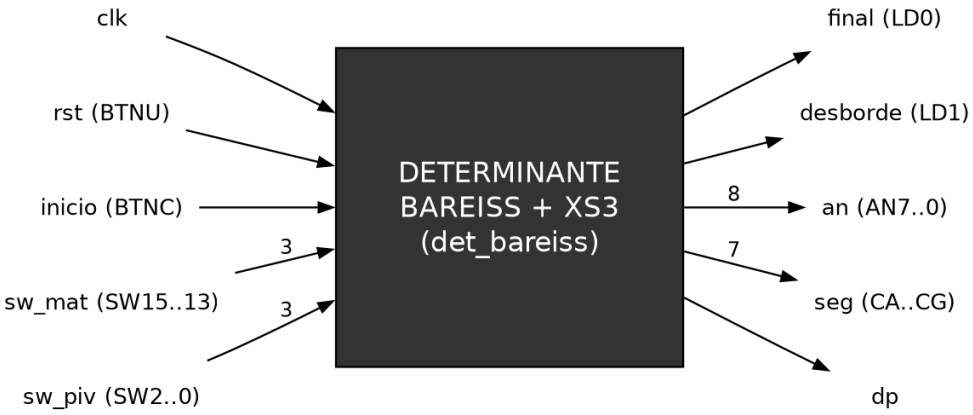


Figura 1. Caja negra del procesador (entidad det_bareiss).

Puerto	Dir.	Bits	Pin Nexys A7	Función
clk	in	1	E3	Reloj de 100 MHz
rst	in	1	BTNU (M18)	Reinicio asíncrono, activo en alto
inicio	in	1	BTNC (N17)	Comienza el cálculo
sw_mat	in	3	SW15..SW13	Elige la matriz de prueba (página de la ROM)
sw_piv	in	3	SW2..SW0	0: muestra el determinante; 1 a 4: muestra el pivote k
final	out	1	LD0	'1' cuando el valor ya está en el display (estado S26)
desborde	out	1	LD1	El valor mostrado tiene más de 7 dígitos
an	out	8	AN7..AN0	Ánodos del display (activos en bajo)
seg	out	7	CA..CG	Segmentos; seg(0) = CA … seg(6) = CG (activos en bajo)
dp	out	1	DP	Punto decimal, siempre apagado

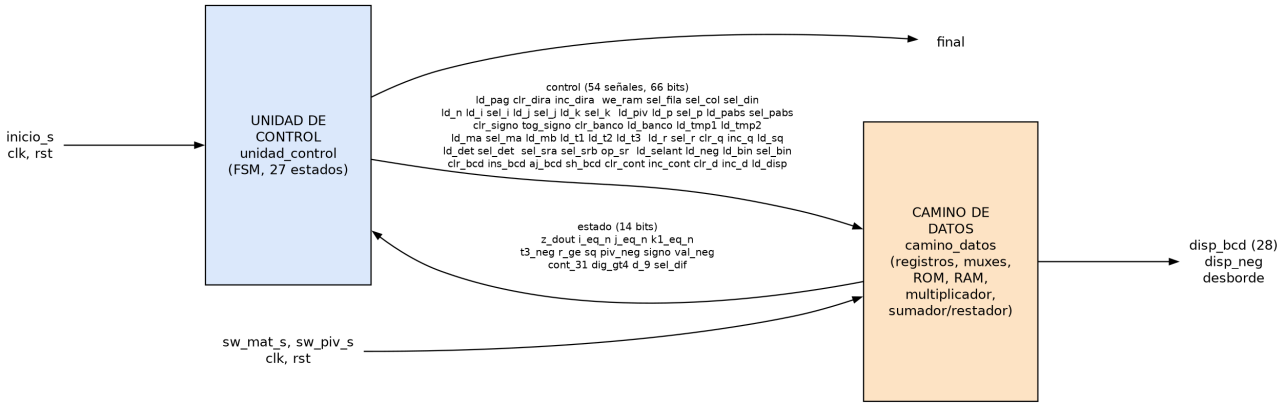


Figura 2. Cajas grises: señales de control (UC → CD) y señales de estado (CD → UC).

La unidad de control no toca datos: solo decide qué hace el camino de datos en cada ciclo. El camino de datos no decide nada: ejecuta las cargas y selecciones que le piden y devuelve las comparaciones que la FSM necesita para elegir el siguiente estado.

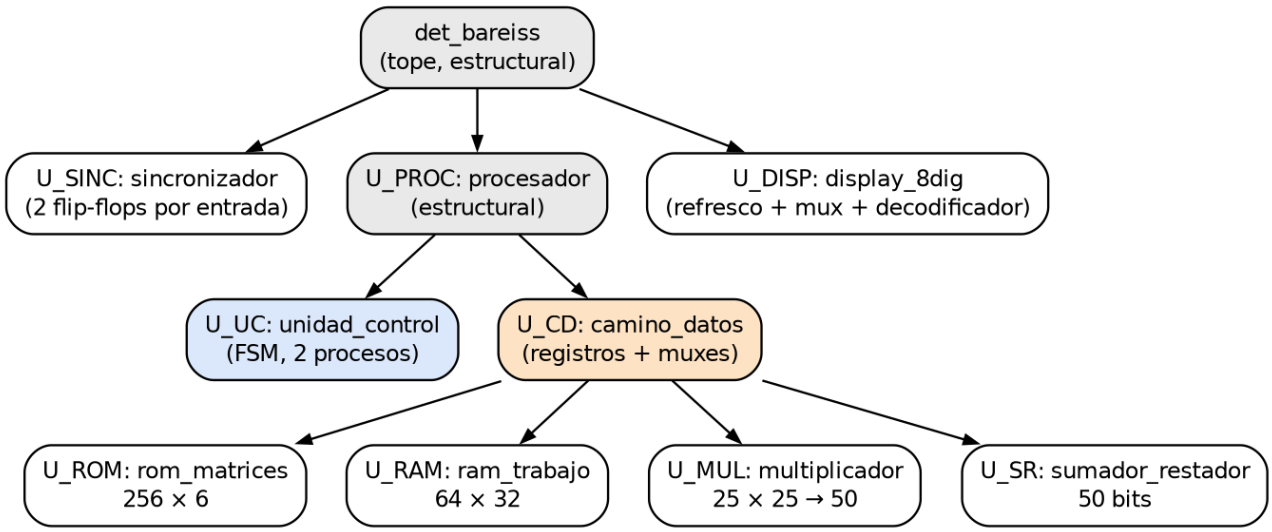


Figura 3. Jerarquía de entidades (lo que muestra el esquemático RTL de ISE).

Entidad	Tipo	Contenido
det_bareiss	Estructural	Tope: sincronizador + procesador + display.
sincronizador	RTL	Dos flip-flops por entrada (inicio, sw_mat, sw_piv) para evitar metaestabilidad.
procesador	Estructural	Unidad de control + camino de datos conectados por las señales de control y de estado.
unidad_control	FSM	Registro de estado (5 bits) y lógica combinacional de estado siguiente y de salidas.
camino_datos	RTL	Registros, multiplexores, comparadores; instancia ROM, RAM, multiplicador y sumador/restador.
rom_matrices	Combinacional	`with dir select` con las 8 matrices de prueba.
ram_trabajo	Memoria	64 × 32 bits, escritura síncrona y lectura asíncrona.
multiplicador	Combinacional	Producto con signo 25 × 25 → 50 bits (`std_logic_signed`).
sumador_restador	Combinacional	50 bits, A + B o A + not(B) + 1.
display_8dig	RTL	Contador de refresco de 18 bits, multiplexor de dígitos y decodificador de 7 segmentos.

5. Pseudocódigo con el hardware disponible

Cada línea indica el estado de la FSM que la ejecuta. Las operaciones del mismo estado ocurren en el mismo ciclo de reloj.

```
S0 :  final = 0;  esperar inicio = 1;  pag = sw_mat
S1 :  n = ROM(00h)
S2 :  para j = 1..n, i = 1..n:  M(i,j) = ROM(dira);  dira = dira + 1     (orden por columnas)
S3 :  k = 1;  p = 1;  |p| = 1;  signo = 0;  banco = 0
mientras k <= n-1:
  S4 :  piv = M(k,k)
        si piv = 0:
  S5 :     buscar i = k+1..n con M(i,k) /= 0
           si no existe:  det = 0;  ir a VISUALIZAR
  S6-S9:   intercambiar filas k e i (columnas k..n);  signo = not signo;  volver a S4
  S10:  banco(k) = piv
        para i = k+1..n, j = k+1..n:
  S11-S14:  t1 = piv · M(i,j);   t2 = M(i,k) · M(k,j)        (un multiplicador, dos usos)
  S15:      t3 = t1 - t2                                     (sumador/restador)
```

```
  S16-S17:  Q = |t3| / |p|  por restas sucesivas            (sumador/restador)
  S18:      M(i,j) = +Q  o  -Q   (signo = signo(t3) xor signo(p))
  S19:  p = piv;  |p| = |piv|;  k = k + 1
S20:  det = M(n,n);  si signo = 1 entonces det = -det
VISUALIZAR:
S21:  val = det si sw_piv = 0, si no banco(sw_piv);  bin = |val|;  neg = signo(val)
S22:  XS3 paso 1:  BCD = 0;  cont = 0
repetir:
  S23:  XS3 paso 2:  BCD(0) = bin(31);   si cont = 31 salir   (paso 5)
  S24:  XS3 paso 3:  para d = 0..9:  si BCD(d) > 4 entonces BCD(d) = BCD(d) + 3
  S25:  XS3 paso 4:  BCD = BCD << 1;  bin = bin << 1;  cont = cont + 1
S26:  final = 1;  display = BCD;  si inicio = 1 ir a S1;  si cambia sw_piv ir a S21
```

Los «para» y «mientras» del pseudocódigo no son lazos del VHDL: cada uno es un lazo de la FSM que vuelve a un estado mientras un contador del camino de datos no llegue a su límite.

6. Diagrama ASM

Convenciones

Elemento	Significado
Caja de estado (rectángulo azul)	Operaciones de todo el estado: salidas Moore de la FSM.
Caja de decisión (rombo)	Pregunta sobre una señal de estado; entre paréntesis va el nombre de la señal en el VHDL. La salida 1 es la condición verdadera.
Caja de salida condicional (rectángulo redondeado naranja)	Operación que solo ocurre si se toma ese camino: salida Mealy de la FSM.
Bloque ASM (recuadro punteado)	Caja de estado más sus decisiones y salidas condicionales: dura un ciclo de reloj.
Conector (círculo verde)	Une hojas del diagrama: A → S1, B → S4, C → S11, D → S21, E → S20, F → S17, G → S22.
x ← expresión	Carga de un registro; se hace efectiva en el flanco de subida al final del ciclo.
x = expresión	Valor combinacional (no se guarda), por ejemplo final = 1 o la selección val.
M(i,j)	Palabra de la RAM en la dirección fila i, columna j.

Todas las decisiones de un bloque usan los valores que tienen los registros al comenzar el ciclo. Por ejemplo, en S19 la pregunta k = n − 1 usa el k anterior al incremento. Cada combinación de estado y decisiones lleva a un solo estado siguiente, y todo camino de decisiones termina en un estado.

Vista general

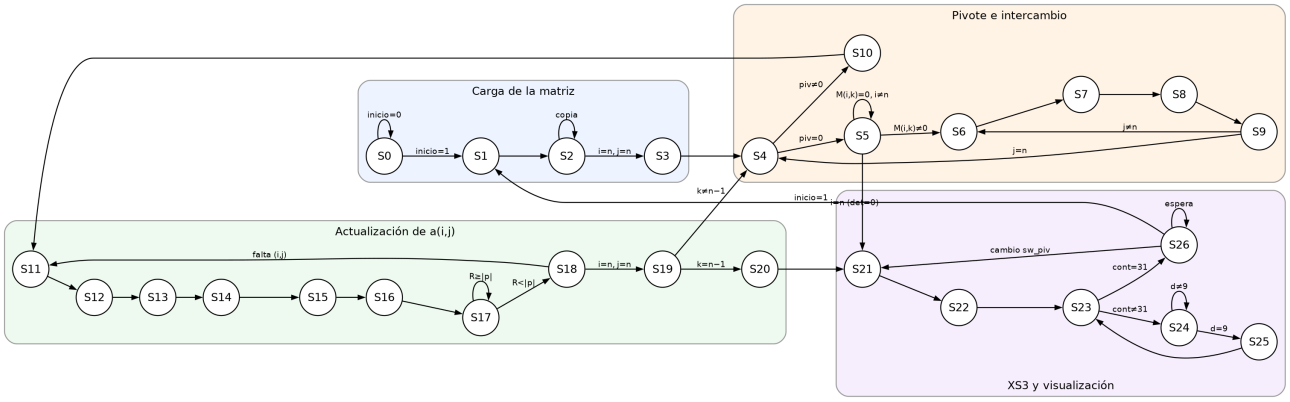


Figura 4. Diagrama de estados de la unidad de control (27 estados), agrupado por función.

Hoja 1: espera y carga de la matriz (S0 a S3)

S2 recorre la ROM con el contador `dira` y los contadores de fila y columna. La dirección de la RAM es fila & columna, así que no hace falta multiplicar para ubicar a(i,j).

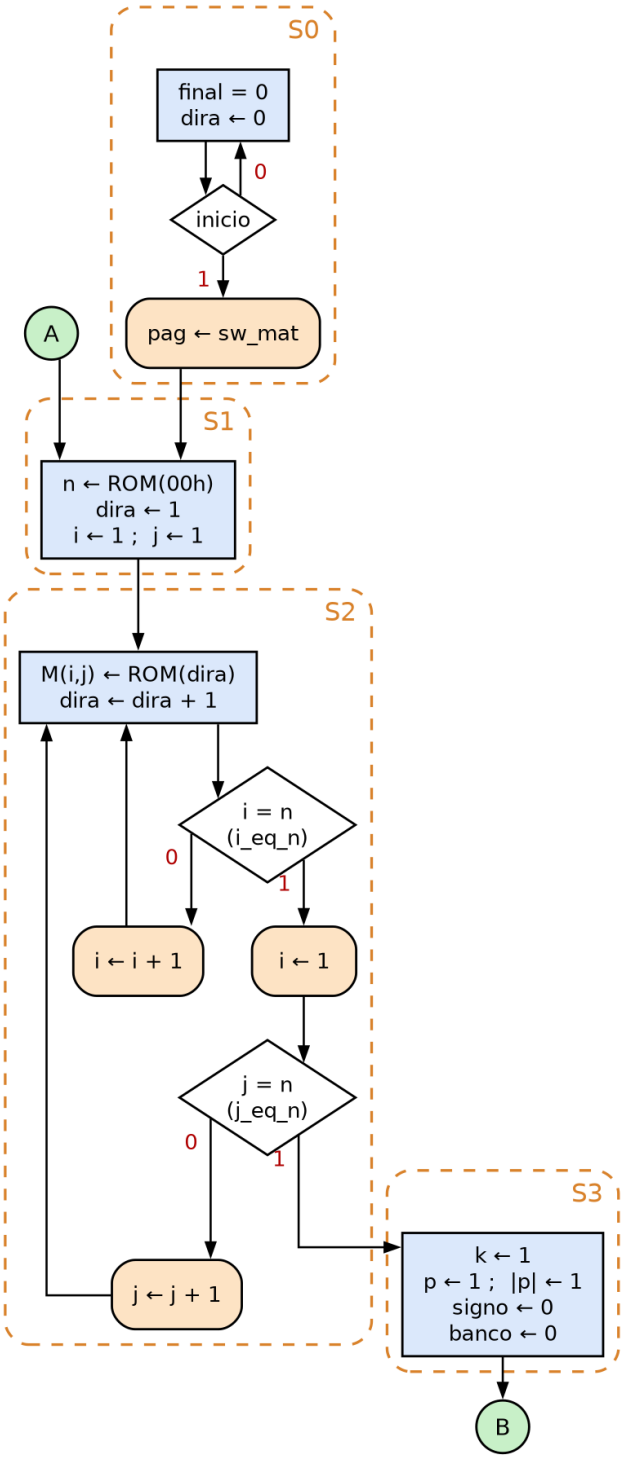


Figura 5. Hoja 1: espera y carga de la matriz (S0 a S3).

Hoja 2: pivote, búsqueda de fila e intercambio (S4 a S10)

La RAM tiene un solo puerto: cada lectura o escritura ocupa un estado, por eso el intercambio usa cuatro estados por columna. Después del intercambio se vuelve a S4 para leer el pivote nuevo.

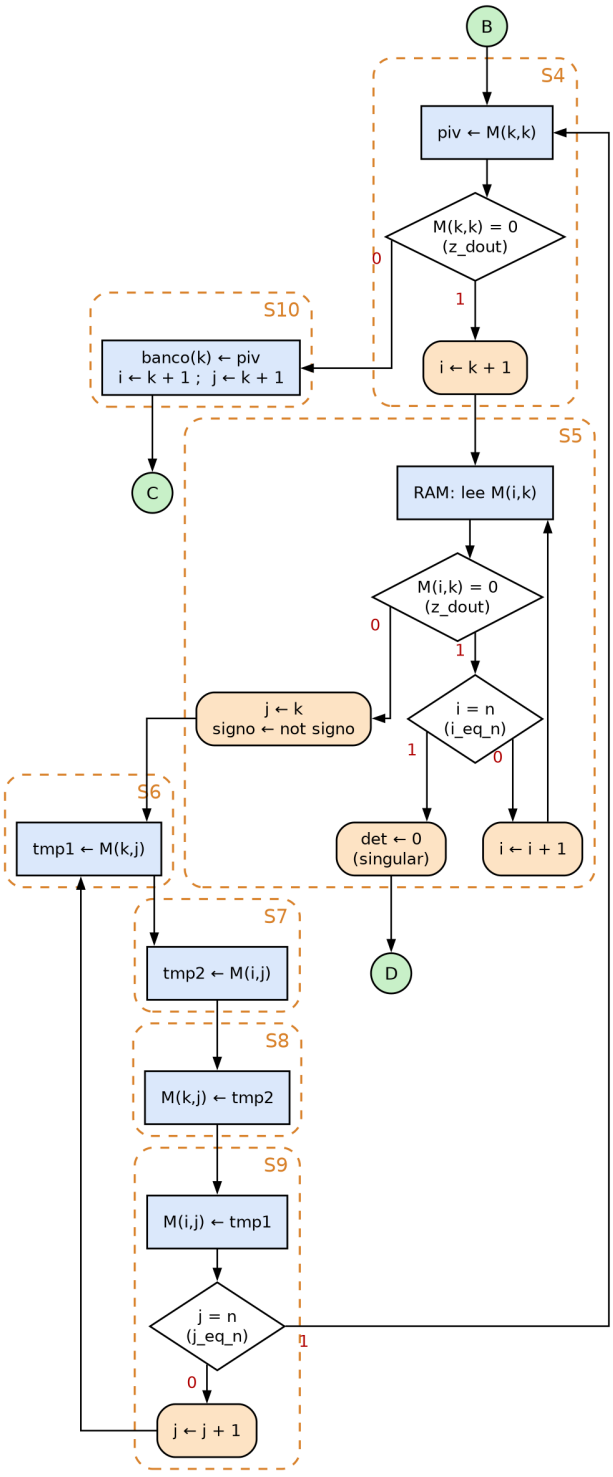


Figura 6. Hoja 2: pivote, búsqueda de fila e intercambio (S4 a S10).

Hoja 3: productos y numerador (S11 a S16)

Con un solo multiplicador, los dos productos de la fórmula (1) se hacen en estados distintos. Los operandos se cargan en mul_a y mul_b un estado antes, para que el camino crítico sea registro → multiplicador → registro.

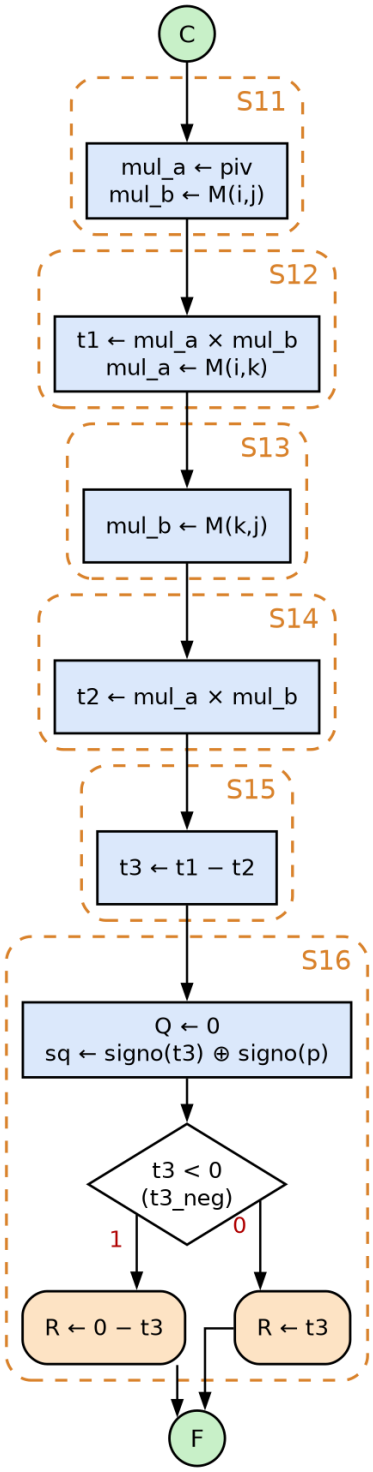


Figura 7. Hoja 3: productos y numerador (S11 a S16).

Hoja 4: división por restas sucesivas y avance (S17 a S19)

S17 es el lazo de la división: el restador calcula R − |p| y la señal r_ge indica si el resultado no es negativo. Si es así, R se carga con la resta y Q se incrementa. El signo se aplica al escribir en S18.

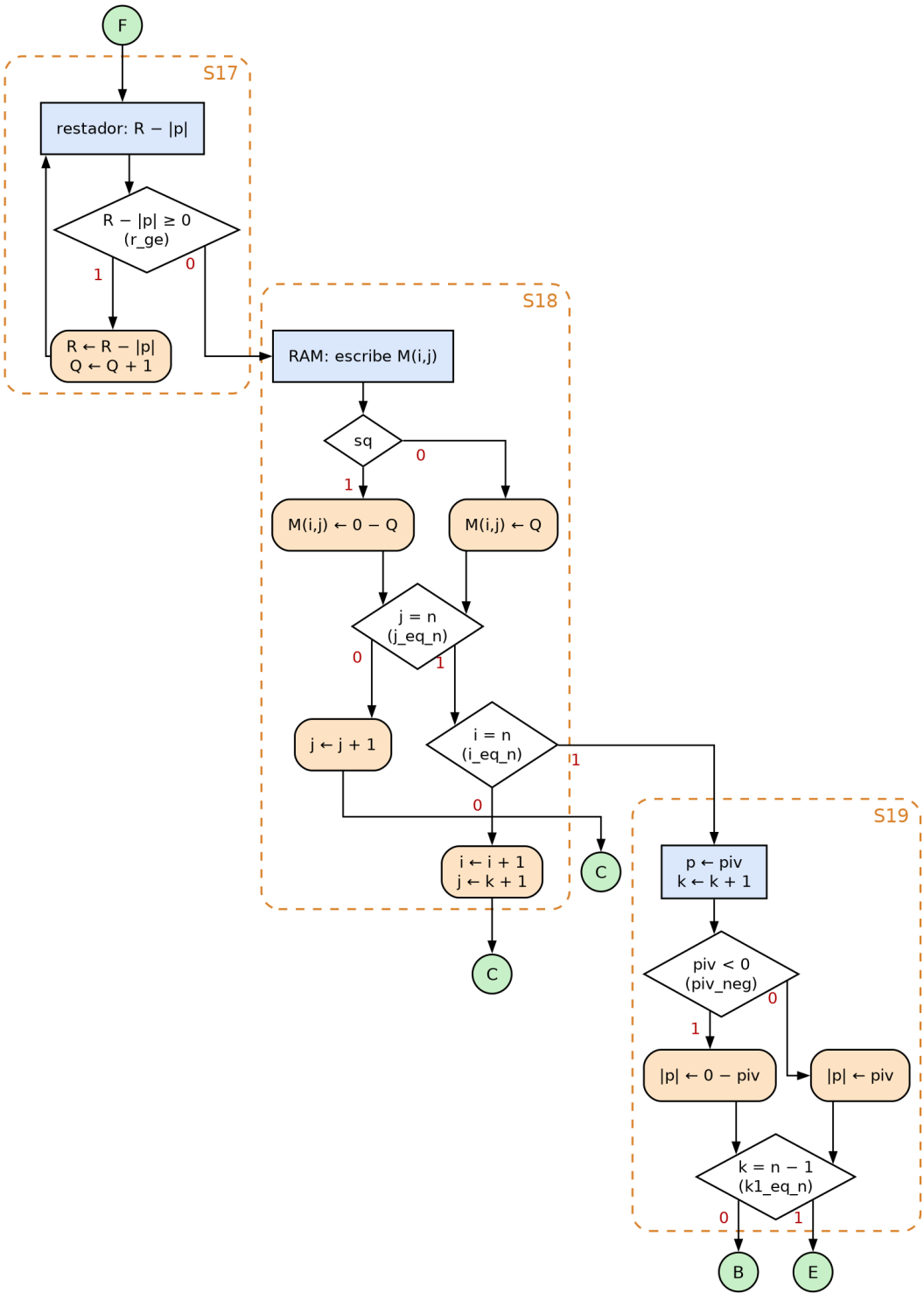


Figura 8. Hoja 4: división por restas sucesivas y avance (S17 a S19).

Hoja 5: determinante y valor a mostrar (S20 y S21)

A S21 se llega desde S20 (cálculo normal), desde S5 (matriz singular) y desde S26 cuando cambian SW2..SW0.

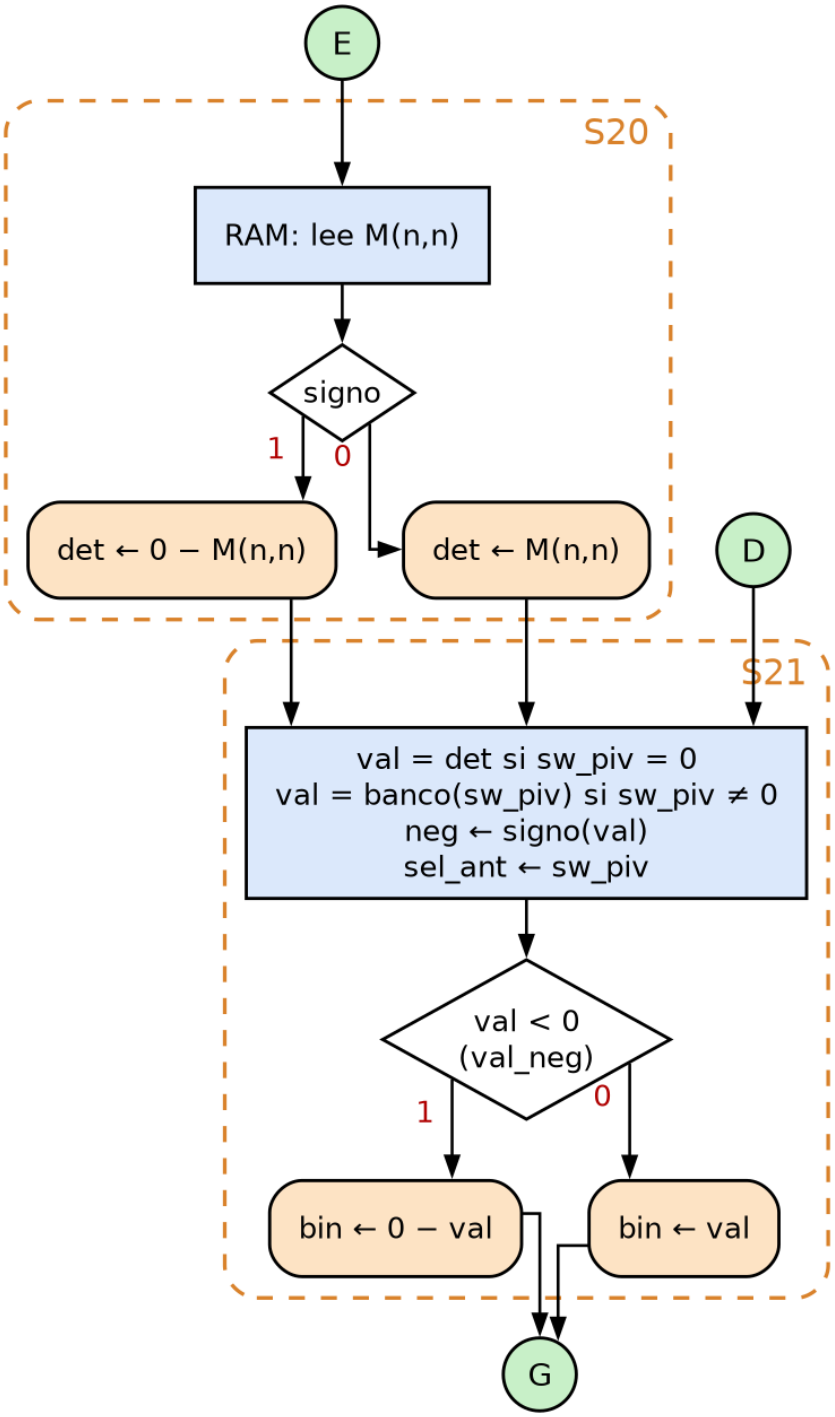


Figura 9. Hoja 5: determinante y valor a mostrar (S20 y S21).

Hoja 6: algoritmo XS3 paso a paso y fin (S22 a S26)

S26 espera con final = 1 y el valor en el display. Si cambian SW2..SW0, la FSM vuelve a S21 y repite el XS3 con el pivote elegido (373 ciclos, invisibles para el ojo).

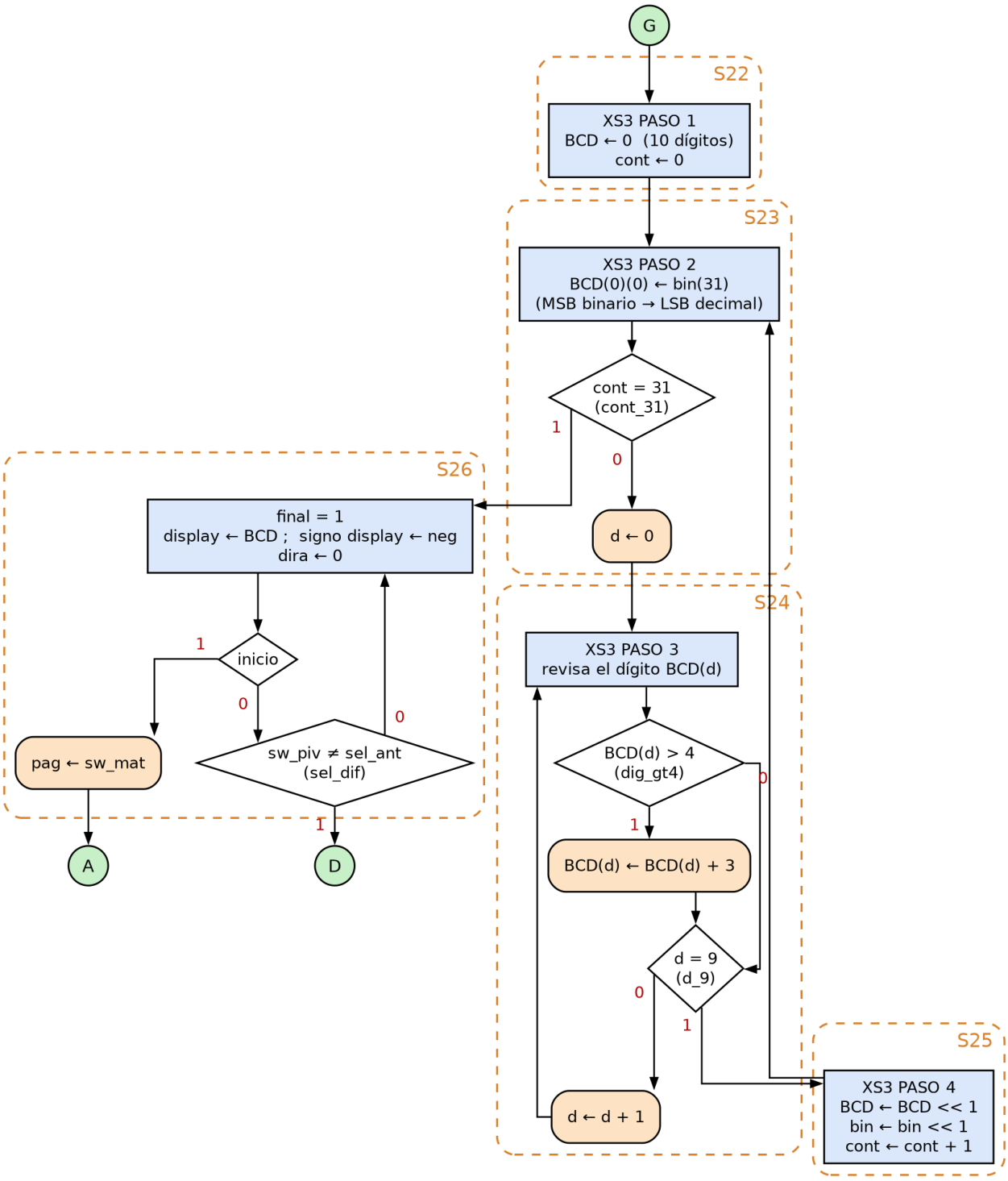


Figura 10. Hoja 6: algoritmo XS3 paso a paso y fin (S22 a S26).

7. Unidad de control

La entidad `unidad_control` tiene dos procesos, el modelo clásico de FSM en RTL:

- **Registro de estado** (proceso secuencial): `st <= prox` en cada flanco de subida; `st <= S0` con rst.
- **Lógica combinacional** (proceso combinacional sensible a st, inicio y las señales de estado): primero pone todas las salidas en su valor de reposo (ninguna carga, `op_sr = '1'`, `prox <= st`) y luego, con `case st is`, activa las señales del estado actual. Las asignaciones dentro de un `if` son las salidas condicionales (Mealy) del ASM. Como todas las salidas tienen un valor por defecto, no se generan latches.

Codificación de estados

Estado	Código	Estado	Código	Estado	Código
S0	00000	S9	01001	S18	10010
S1	00001	S10	01010	S19	10011
S2	00010	S11	01011	S20	10100
S3	00011	S12	01100	S21	10101
S4	00100	S13	01101	S22	10110
S5	00101	S14	01110	S23	10111
S6	00110	S15	01111	S24	11000
S7	00111	S16	10000	S25	11001
S8	01000	S17	10001	S26	11010

El código es binario de 5 bits (`constant S0 : STD_LOGIC_VECTOR(4 downto 0) := "00000"`). XST puede recodificar la FSM (por ejemplo a one-hot); el reporte de síntesis indica la codificación final.

Palabra de control por estado

Esta tabla se generó leyendo el `case st is` del VHDL. La columna Moore lista las señales que se activan en todo el estado. La columna Mealy muestra las que dependen de una señal de estado y el estado siguiente (→). Las señales que no aparecen están en reposo: '0', selección "00"/"000" y `op_sr = 1`.

Estado	Código	Salidas Moore	Salidas Mealy y estado siguiente
S0	00000	clr_dira	si inicio: ld_pag, → S1 si no: → S0
S1	00001	ld_n, inc_dira, ld_i, sel_i = 00, ld_j, sel_j = 00	→ S2
S2	00010	we_ram, sel_fila = 00, sel_col = 00, sel_din = 000, inc_dira	si i_eq_n: ld_i, sel_i = 00 si j_eq_n: → S3 si no: ld_j, sel_j = 01, → S2 si no: ld_i, sel_i = 01, → S2
S3	00011	ld_k, sel_k = 0, ld_p, sel_p = 0, ld_pabs, sel_pabs = 00, clr_signo, clr_banco	→ S4

Estado	Código	Salidas Moore	Salidas Mealy y estado siguiente
S4	00100	sel_fila = 01, sel_col = 01, ld_piv	si z_dout: ld_i, sel_i = 10, → S5 si no: → S10
S5	00101	sel_fila = 00, sel_col = 01	si no z_dout: ld_j, sel_j = 11, tog_signo, → S6 si no, si i_eq_n: ld_det, sel_det = 00, → S21 si no: ld_i, sel_i = 01, → S5
S6	00110	sel_fila = 01, sel_col = 00, ld_tmp1	→ S7
S7	00111	sel_fila = 00, sel_col = 00, ld_tmp2	→ S8
S8	01000	we_ram, sel_fila = 01, sel_col = 00, sel_din = 010	→ S9
S9	01001	we_ram, sel_fila = 00, sel_col = 00, sel_din = 001	si j_eq_n: → S4 si no: ld_j, sel_j = 01, → S6
S10	01010	ld_banco, ld_i, sel_i = 10, ld_j, sel_j = 10	→ S11
S11	01011	sel_fila = 00, sel_col = 00, ld_ma, sel_ma = 0, ld_mb	→ S12
S12	01100	sel_fila = 00, sel_col = 01, ld_t1, ld_ma, sel_ma = 1	→ S13
S13	01101	sel_fila = 01, sel_col = 00, ld_mb	→ S14
S14	01110	ld_t2	→ S15
S15	01111	sel_sra = 01, sel_srb = 000, op_sr = 1, ld_t3	→ S16
S16	10000	sel_sra = 00, sel_srb = 001, op_sr = 1, clr_q, ld_sq, ld_r	si t3_neg: sel_r = 1 si no: sel_r = 0 → S17
S17	10001	sel_sra = 10, sel_srb = 010, op_sr = 1	si r_ge: ld_r, sel_r = 1, inc_q, → S17 si no: → S18
S18	10010	sel_sra = 00, sel_srb = 011, op_sr = 1, we_ram, sel_fila = 00, sel_col = 00	si sq: sel_din = 100 si no: sel_din = 011 si j_eq_n: si i_eq_n: → S19 si no: ld_i, sel_i = 01, ld_j, sel_j = 10, → S11 si no: ld_j, sel_j = 01, → S11
S19	10011	sel_sra = 00, sel_srb = 100, op_sr = 1, ld_p, sel_p = 1, ld_pabs, ld_k, sel_k = 1	si piv_neg: sel_pabs = 10 si no: sel_pabs = 01 si k1_eq_n: → S20 si no: → S4

Estado	Código	Salidas Moore	Salidas Mealy y estado siguiente
S20	10100	sel_fila = 10, sel_col = 10, sel_sra = 00, sel_srb = 101, op_sr = 1, ld_det	si signo: sel_det = 10 si no: sel_det = 01 → S21
S21	10101	sel_sra = 00, sel_srb = 110, op_sr = 1, ld_selant, ld_neg, ld_bin	si val_neg: sel_bin = 01 si no: sel_bin = 00 → S22
S22	10110	clr_bcd, clr_cont	→ S23
S23	10111	ins_bcd	si cont_31: → S26 si no: clr_d, → S24
S24	11000	sel_sra = 11, sel_srb = 111, op_sr = 0	si dig_gt4: aj_bcd si d_9: → S25 si no: inc_d, → S24
S25	11001	sh_bcd, ld_bin, sel_bin = 10, inc_cont	→ S23
S26	11010	final, ld_disp, clr_dira	si inicio: ld_pag, → S1 si no, si sel_dif: → S21 si no: → S26

8. Señales del procesador

8.1 Señales de estado (CD → UC)

Señal	Lógica en el camino de datos	Se usa en
inicio	Botón BTNC tras el sincronizador	S0, S26
z_dout	dout = 0 (comparador de 32 bits con la salida de la RAM)	S4, S5
i_eq_n	i = n (comparador de 3 bits)	S2, S5, S18
j_eq_n	j = n	S2, S9, S18
k1_eq_n	k + 1 = n, es decir k = n − 1	S19
t3_neg	t3(49), bit de signo del numerador	S16
r_ge	not sr_s(49): la resta R − |p| no es negativa	S17
sq	Registro sq_r = signo(t3) xor signo(p)	S18
piv_neg	piv(31)	S19
signo	Registro signo_r: paridad de intercambios	S20
val_neg	val(31), signo del valor seleccionado	S21
cont_31	cont = 11111	S23
dig_gt4	BCD(d) > 0100 (comparador de 4 bits a la salida del mux de dígitos)	S24
d_9	d = 1001	S24
sel_dif	sw_piv ≠ sel_ant	S26

8.2 Señales de control (UC → CD)

Señal	Bits	Efecto en el camino de datos
ld_pag	1	pag ← sw_mat
clr_dira, inc_dira	1, 1	dira ← 0 / dira ← dira + 1
we_ram	1	Escribe din en RAM(fila & col)
sel_fila	2	Fila de la RAM: 00 = i, 01 = k, 10 = n
sel_col	2	Columna de la RAM: 00 = j, 01 = k, 10 = n
sel_din	3	Dato de la RAM: 000 = ROM extendida, 001 = tmp1, 010 = tmp2, 011 = Q, 100 = sr_s (−Q)
ld_n	1	n ← ROM(2..0)
ld_i, sel_i	1, 2	i ← 00: 1, 01: i + 1, 10: k + 1
ld_j, sel_j	1, 2	j ← 00: 1, 01: j + 1, 10: k + 1, 11: k
ld_k, sel_k	1, 1	k ← 0: 1, 1: k + 1
ld_piv	1	piv ← dout
ld_p, sel_p	1, 1	p ← 0: 1, 1: piv
ld_pabs, sel_pabs	1, 2	|p| ← 00: 1, 01: piv, 10: sr_s (−piv)
clr_signo, tog_signo	1, 1	signo ← 0 / signo ← not signo
clr_banco, ld_banco	1, 1	b1..b4 ← 0 / b(k) ← piv
ld_tmp1, ld_tmp2	1, 1	tmp1 ← dout / tmp2 ← dout
ld_ma, sel_ma, ld_mb	1, 1, 1	mul_a ← 0: piv, 1: dout / mul_b ← dout
ld_t1, ld_t2, ld_t3	1, 1, 1	t1 ← producto / t2 ← producto / t3 ← sr_s
ld_r, sel_r	1, 1	R ← 0: t3, 1: sr_s
clr_q, inc_q, ld_sq	1, 1, 1	Q ← 0 / Q ← Q + 1 / sq ← signo(t3) xor signo(p)

Señal	Bits	Efecto en el camino de datos
ld_det, sel_det	1, 2	det ← 00: 0, 01: dout, 10: sr_s (−M(n,n))
sel_sra	2	Operando A del sumador: 00 = 0, 01 = t1, 10 = R, 11 = BCD(d)
sel_srb	3	Operando B: 000 = t2, 001 = t3, 010 = |p|, 011 = Q, 100 = piv, 101 = dout, 110 = val, 111 = 3
op_sr	1	0 = suma, 1 = resta
ld_selant, ld_neg	1, 1	sel_ant ← sw_piv / neg ← signo(val)
ld_bin, sel_bin	1, 2	bin ← 00: val, 01: sr_s (−val), 10: bin << 1
clr_bcd, ins_bcd, aj_bcd, sh_bcd	1, 1, 1, 1	XS3 pasos 1, 2, 3 y 4 sobre el registro BCD
clr_cont, inc_cont	1, 1	cont ← 0 / cont ← cont + 1
clr_d, inc_d	1, 1	d ← 0 / d ← d + 1
ld_disp	1	disp ← BCD; disp_neg ← neg
final	1	Salida del procesador (LD0)

8.3 Registros y tamaños

Registro	Bits	Uso
st (en la UC)	5	Estado de la FSM
pag, dira	3, 5	Página y dirección de la ROM
n, i, j, k	3 c/u	Tamaño de la matriz y contadores de fila, columna y paso
piv, p, |p|	32 c/u	Pivote actual, pivote anterior (divisor) y su magnitud
signo_r	1	Paridad de intercambios de filas
b1, b2, b3, b4	32 c/u	Banco de pivotes
tmp1, tmp2	32 c/u	Intercambio de filas
mul_a, mul_b	25 c/u	Registros de entrada del multiplicador
t1, t2, t3	50 c/u	Productos y numerador de la fórmula (1)
R, Q, sq_r	50, 32, 1	Residuo, cociente y signo del cociente
det	32	Determinante
sel_ant, neg	3, 1	Selección ya convertida y signo del valor convertido
bin, BCD	32, 40	Registros binario y decimal del XS3
cont, d	5, 4	Bits procesados y dígito revisado en el XS3
disp, disp_neg_r	40, 1	Valor que se ve en el display
refresco (display_8dig)	18	Barrido del display: 2^{15} ciclos por dígito, 2,6 ms por barrido

Por qué estos anchos

Por la identidad de Sylvester, cada valor guardado en la RAM es el determinante de una submatriz m×m de A. La desigualdad de Hadamard lo acota: $|\det| \le (32\cdot\sqrt{m})^m$ cuando $|a(i,j)| \le 32$.

- Operandos del multiplicador: en el paso k son submatrices de k×k con k ≤ 4, así que $|x| < 2^{24} =$ 16 777 216. Caben en 25 bits con signo. (La cota no se alcanza porque +32 no existe en 6 bits.)
- Determinante 5×5: $|\det| \le 1{,}88 \times 10^9 < 2^{31}$. Por eso la RAM, det, Q y bin son de 32 bits.
- Numerador: t3 = p · a(k), con |p| ≤ 170 268 (submatriz 3×3), así que $|t3| \le 3{,}2 \times 10^{14} < 2^{49}$. t1, t2, t3, R y el sumador/restador son de 50 bits.
- BCD: 32 bits binarios necesitan 10 dígitos decimales. El display muestra los 7 de abajo; si los dígitos 7 a 9 no son cero, LD1 (desborde) se enciende.

9. Camino de datos

El camino de datos se organiza alrededor de dos buses: la salida de la RAM (`dout`) y la salida del sumador/restador (`sr_s`). Entre corchetes están las señales de control de cada bloque.

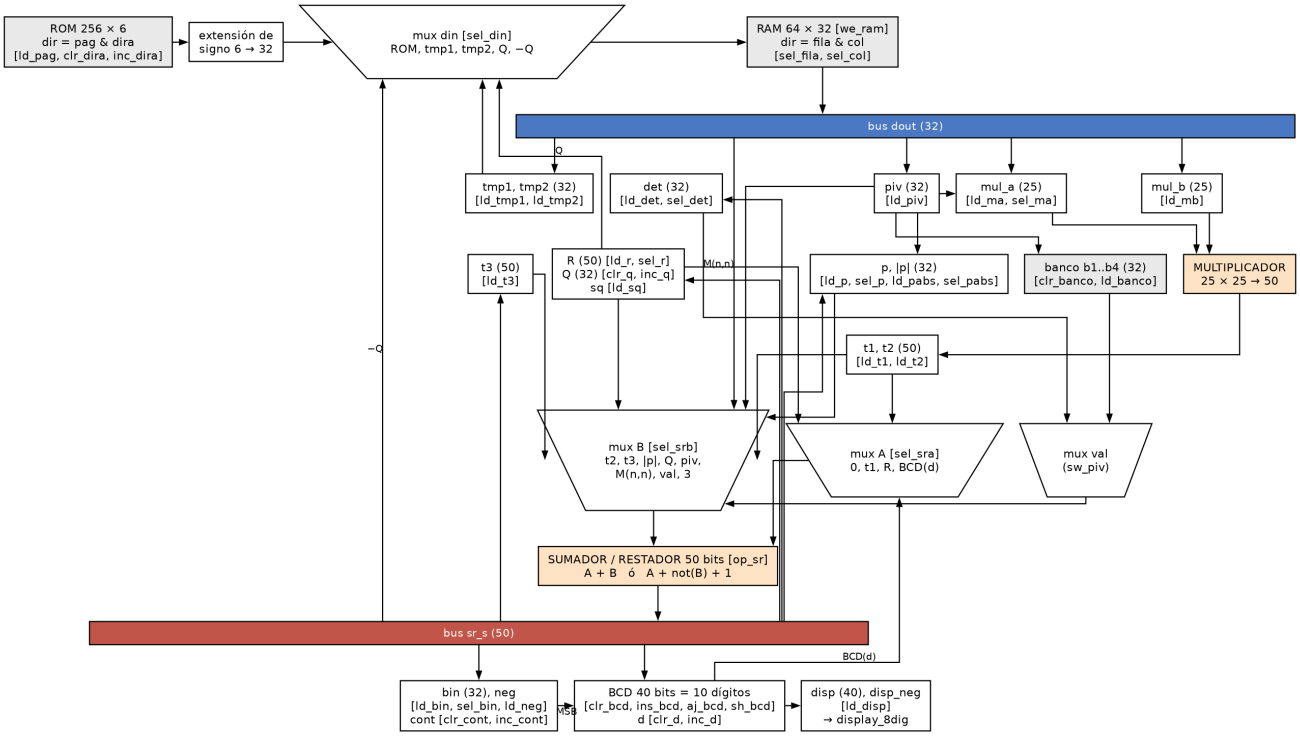


Figura 11. Camino de datos con sus señales de control.

Cómo está escrito cada elemento

Elemento	Forma en el VHDL
Registro con carga	`if ld_x = '1' then x <= x_d; end if;` dentro del único proceso con reloj del camino de datos.
Contador	`if clr_x = '1' then x <= 0 elsif inc_x = '1' then x <= x + '1'.`
Multiplexor	`with sel_x select x_d <= … when "00", …` (sentencia concurrente).
Comparador	`z <= '1' when a = b else '0';`
Extensión de signo	Señal auxiliar `sx <= (others => x(31));` y concatenación `sx & x`.
Unidad funcional	Entidad combinacional instanciada con `port map` (multiplicador, sumador/restador).
Memoria	ROM: `with dir select`. RAM: arreglo con escritura en el flanco y lectura asíncrona.

La síntesis genérica con GHDL (`ghdl --synth`) confirma un solo multiplicador (25 × 25), un solo sumador de 50 bits, la RAM de 64 × 32 inferida como memoria y ningún latch.

10. Tiempo de ejecución

- Carga de la matriz (S0 a S3): $n^2 + 3$ ciclos.
- Paso k: S4, S10 y S19 (3 ciclos) más, por cada uno de los $(n - k)^2$ elementos, 8 ciclos fijos (S11 a S16 y S18) y $|\text{cociente}| + 1$ ciclos de división en S17.
- Intercambio de filas: hasta $n - k$ ciclos de búsqueda, 4 ciclos por columna y una nueva lectura del pivote.

- XS3: 373 ciclos desde S21 hasta llegar a S26.

La división por restas sucesivas domina el tiempo cuando los valores son grandes, porque tarda tantos ciclos como vale el cociente. En la página 7 (5×5 con valores cercanos a ±32) el cálculo tarda 19 013 425 ciclos, 0,19 s a 100 MHz. En el peor caso teórico el último cociente se acerca a 1,9 × 10^9, unos 19 s. Una división con corrimiento y resta (restauración) haría lo mismo en 50 ciclos; aquí se dejó la de restas sucesivas porque es la que pide el enunciado.

11. Implementación en ISE 14.7 y la Nexys A7

11.1 Proyecto

- File → New Project. Top-level source type: HDL.
- Family: Artix7 · Device: XC7A100T · Package: CSG324 · Speed: −1 · Synthesis: XST · Simulator: ISim · Preferred language: VHDL. ISE solo soporta la Nexys A7-100T; la variante 50T exige Vivado.
- Add Source: src/det_bareiss.vhd y src/nexys_a7.ucf (Implementation); sim/tb_procesador.vhd y sim/tb_det_bareiss.vhd (Simulation). ISE detecta det_bareiss como tope; si no, márquenlo con «Set as Top Module».
- Synthesize → View RTL Schematic para ver la jerarquía de la figura 3. Luego Implement Design y Generate Programming File. Revisen en el reporte de tiempos que se cumpla TS_sys_clk (10 ns).
- Programación: según el manual de la tarjeta, la Nexys A7 no funciona con Digilent Adept. Usen iMPACT de ISE o el Hardware Manager de Vivado (Lab Edition) con el archivo .bit.

11.2 Operación en la tarjeta

- Pulsar BTNU (rst).
- Elegir la matriz con SW15..SW13 (0 a 7, tabla 11.3) y dejar SW2..SW0 en 000.
- Pulsar BTNC (inicio). Cuando termina, LD0 (final) se enciende y el display muestra el signo en AN7 y 7 dígitos.
- Poner SW2..SW0 en 1 a n − 1 para ver cada pivote; volver a 000 para ver el determinante.
- Si LD1 (desborde) está encendido, el valor tiene más de 7 dígitos y el display muestra los 7 de abajo.
- Para calcular otra matriz: cambiar SW15..SW13 y pulsar BTNC de nuevo.

11.3 Matrices de prueba en la ROM

Pág.	n	Matriz A	det	Pivotes	Qué prueba
0	2	7 -3 -3 5	26	7	Caso básico 2×2
1	3	2 1 3 1 4 2 3 2 5	3	2, 7	Ejemplo del documento
2	4	0 2 1 3 2 1 0 1 1 0 3 2 3 1 2 4	-22	2, 4, 13	a11 = 0: intercambio en k = 1
3	5	4 -2 1 3 0 -2 6 2 -1 5 1 2 -7 4 2 3 -1 4 5 -3 0 5 2 -3 8	362	4, 20, -170, -639	5×5 general, pivotes negativos

Pág.	n	Matriz A	det	Pivotes	Qué prueba
4	3	-2 4 1 4 -8 3 1 3 5	50	-2, -10	Pivote negativo; intercambio en k = 2
5	4	1 2 3 0 2 4 1 5 3 1 -2 4 0 5 4 -3	150	1, -5, 25	Pivote cero en k = 2 (p ≠ 1)
6	5	0 0 0 0 0 0 3 1 2 4 0 1 5 -2 1 0 2 -2 6 3 0 4 1 3 7	0	—	Columna 1 en cero: singular, det = 0
7	5	30 27 24 -29 -31 27 -21 28 29 -24 24 28 2 -28 -28 -29 29 -28 -27 28 -31 -24 -28 28 -30	-17 121 828	30, -1359, 22146, 277564	Valores extremos; desborde del display

Para cambiar o agregar matrices, editen la lista MATRICES en sim/modelo_referencia.py y ejecútenlo: imprime el det y los pivotes esperados y las líneas de la ROM (formato `"dato" when "dirección",`) listas para pegar en la entidad rom_matrices.

11.4 Pines (src/nexys_a7.ucf)

Señal	Pines
clk	E3 (100 MHz)
rst / inicio	M18 (BTNU) / N17 (BTNC)
sw_piv(0..2)	J15 (SW0), L16 (SW1), M13 (SW2)
sw_mat(0..2)	U12 (SW13), U11 (SW14), V10 (SW15)
final / desborde	H17 (LD0) / K15 (LD1)
seg(0..6) = CA..CG, dp	T10, R10, K16, K13, P15, T11, L18; DP = H15
an(0..7)	J17, J18, T9, J14, P14, T14, K2, U13

12. Simulación

- **sim/tb_procesador.vhd** prueba la unidad de control con el camino de datos. Para cada matriz pulsa inicio, espera final = 1 y compara disp_bcd, disp_neg y desborde con el valor esperado; después recorre los pivotes con sw_piv = 1 a n − 1. El genérico MAX_MATRIZ del testbench vale 6 por defecto (unos 0,2 ms simulados); con 7 incluye la matriz extrema, que tarda bastante.
- **sim/tb_det_bareiss.vhd** prueba el tope completo con el sincronizador y el display: calcula la página 3 y lee los 8 dígitos decodificando an y seg durante un barrido completo (2^{18} ciclos). Luego elige el pivote 3.

Pág.	Tamaño	Display esperado	Display simulado	LD1	Ciclos	Resultado
0	2x2	' 0000026'	' 0000026'	0	415	OK
1	3x3	' 0000003'	' 0000003'	0	442	OK
2	4x4	'-0000022'	'-0000022'	0	627	OK
3	5x5	' 0000362'	' 0000362'	0	4 203	OK
4	3x3	' 0000050'	' 0000050'	0	520	OK
5	4x4	' 0000150'	' 0000150'	0	807	OK

Pág.	Tamaño	Display esperado	Display simulado	LD1	Ciclos	Resultado
6	5x5	' 0000000'	' 0000000'	0	403	OK
7	5x5	'-7121828'	'-7121828'	1	19 013 425	OK

Todos los pivotes también coinciden, y el testbench del tope leyó « 0000362» y «-0000170» en los segmentos (ver sim/resultado_ghdl.txt). Para comprobar que los testbench detectan errores se probaron dos fallas en una copia del diseño: con «+2» en lugar de «+3» en el XS3, y con |p| ← piv en lugar de |p| ← −piv en S19. En ambos casos el testbench reportó errores.

13. Limitaciones y verificación pendiente

- **No se ejecutó ISE.** El diseño se verificó con GHDL 4.1 (simulación con `-fsynopsys -fexplicit` y síntesis genérica), no con XST ni ISim. Hay que sintetizar en ISE 14.7 y revisar el reporte de tiempos a 100 MHz. Si no se cumpliera, generen un reloj de 50 MHz con el Clocking Wizard y pongan PERIOD = 20 ns.
- **Arreglo de la RAM.** Es la única construcción que no es std_logic ni std_logic_vector. Si el profesor no la acepta, reemplacen la entidad ram_trabajo por el modelo de RAM del aula con los mismos puertos.
- **Modelo de RAM.** Se usó lectura asíncrona. Si el modelo del aula tiene lectura síncrona, cada lectura (S4, S5, S6, S7, S11, S12, S13, S20) necesita un estado de espera antes de usar el dato.
- **Display de 7 dígitos.** Con signo en AN7 caben valores hasta ±9 999 999; para valores mayores se enciende LD1 y se ven los 7 dígitos de abajo.
- **Tiempo de la división.** Proporcional al cociente (sección 10).
- **Valor de n.** El diseño supone 2 ≤ n ≤ 5 en la ROM y no lo valida.
- **ROM con 8 páginas.** La selección de página con SW15..SW13 permite mostrar varios tamaños sin volver a sintetizar. Si el profesor quiere una sola matriz, usen la página 0 con esos switches en 0.

Anexo A. Código VHDL completo (src/det_bareiss.vhd)

```
 1  ----------------------------------------------------------------------------------
 2  -- Universidad Distrital Francisco Jose de Caldas
 3  -- Analisis y Diseno de Microprocesadores - Taller 5
 4  --
 5  -- Microprocesador de proposito especifico: determinante de una matriz simetrica
 6  -- de 2x2 a 5x5 por el algoritmo de Bareiss, division por restas sucesivas y
 7  -- conversion binario -> BCD por el algoritmo XS3 (paso a paso en el ASM).
 8  --
 9  -- Herramienta : Xilinx ISE 14.7 (XST / ISim)
10  -- Tarjeta     : Digilent Nexys A7-100T (XC7A100T-1CSG324C), reloj de 100 MHz
11  --
12  -- Estilo RTL:
13  --   * Solo senales de tipo std_logic y std_logic_vector (sin variables, sin
14  --     integer, sin signed/unsigned, sin genericos).  La unica excepcion es el
15  --     arreglo de la RAM (array of std_logic_vector), que es la forma de
16  --     describir una memoria para que XST la infiera.
17  --   * Unidad de control (FSM de 27 estados) separada del camino de datos.
18  --   * Camino de datos = registros con habilitacion + multiplexores + unidades
19  --     funcionales (multiplicador, sumador/restador) + memorias (ROM, RAM).
20  --   * Sin lazos for ni descripciones tipo software: las repeticiones del
21  --     algoritmo las hace la FSM, un paso por ciclo de reloj.
22  --
23  -- Jerarquia (todas las entidades estan en este archivo):
24  --   det_bareiss (tope)
25  --     |- sincronizador
26  --     |- procesador
27  --     |    |- unidad_control      (FSM / ASM)
28  --     |    |- camino_datos
29  --     |         |- rom_matrices
30  --     |         |- ram_trabajo
31  --     |         |- multiplicador
32  --     |         |- sumador_restador
33  --     |- display_8dig
34  ----------------------------------------------------------------------------------
35
36
37  ----------------------------------------------------------------------------------
38  -- ROM DE MATRICES: 256 x 6 bits, lectura combinacional.
39  -- dir = pag(3) & dira(5). En cada pagina: 00h = n, luego los a(i,j) por columnas
40  -- (a11, a21, ..., an1, a12, ...), como en la Figura 1 del taller.
41  -- Datos en complemento a 2 (-32 a 31).
42  ----------------------------------------------------------------------------------
43  library IEEE;
44  use IEEE.STD_LOGIC_1164.ALL;
45
46  entity rom_matrices is
47      Port ( dir : in  STD_LOGIC_VECTOR (7 downto 0);
48             a   : out STD_LOGIC_VECTOR (5 downto 0));
49  end rom_matrices;
50
51  architecture RTL of rom_matrices is
52  begin
53  with dir select
54      a <= "000010" when "00000000",   -- pag 0, 00h: n = 2
55           "000111" when "00000001",   -- pag 0, 01h: a11 = 7
56           "111101" when "00000010",   -- pag 0, 02h: a21 = -3
57           "111101" when "00000011",   -- pag 0, 03h: a12 = -3
58           "000101" when "00000100",   -- pag 0, 04h: a22 = 5
59           "000011" when "00100000",   -- pag 1, 00h: n = 3
60           "000010" when "00100001",   -- pag 1, 01h: a11 = 2
61           "000001" when "00100010",   -- pag 1, 02h: a21 = 1
62           "000011" when "00100011",   -- pag 1, 03h: a31 = 3
63           "000001" when "00100100",   -- pag 1, 04h: a12 = 1
64           "000100" when "00100101",   -- pag 1, 05h: a22 = 4
65           "000010" when "00100110",   -- pag 1, 06h: a32 = 2
66           "000011" when "00100111",   -- pag 1, 07h: a13 = 3
67           "000010" when "00101000",   -- pag 1, 08h: a23 = 2
68           "000101" when "00101001",   -- pag 1, 09h: a33 = 5
69           "000100" when "01000000",   -- pag 2, 00h: n = 4
70           "000000" when "01000001",   -- pag 2, 01h: a11 = 0
71           "000010" when "01000010",   -- pag 2, 02h: a21 = 2
72           "000001" when "01000011",   -- pag 2, 03h: a31 = 1
73           "000011" when "01000100",   -- pag 2, 04h: a41 = 3
74           "000010" when "01000101",   -- pag 2, 05h: a12 = 2
75           "000001" when "01000110",   -- pag 2, 06h: a22 = 1
76           "000000" when "01000111",   -- pag 2, 07h: a32 = 0
77           "000001" when "01001000",   -- pag 2, 08h: a42 = 1
78           "000001" when "01001001",   -- pag 2, 09h: a13 = 1
79           "000000" when "01001010",   -- pag 2, 0Ah: a23 = 0
80           "000011" when "01001011",   -- pag 2, 0Bh: a33 = 3
81           "000010" when "01001100",   -- pag 2, 0Ch: a43 = 2
82           "000011" when "01001101",   -- pag 2, 0Dh: a14 = 3
83           "000001" when "01001110",   -- pag 2, 0Eh: a24 = 1
84           "000010" when "01001111",   -- pag 2, 0Fh: a34 = 2
85           "000100" when "01010000",   -- pag 2, 10h: a44 = 4
```

```
          "000101" when "01100000",   -- pag 3, 00h: n = 5
          "000100" when "01100001",   -- pag 3, 01h: a11 = 4
          "111110" when "01100010",   -- pag 3, 02h: a21 = -2
          "000001" when "01100011",   -- pag 3, 03h: a31 = 1
          "000011" when "01100100",   -- pag 3, 04h: a41 = 3
          "000000" when "01100101",   -- pag 3, 05h: a51 = 0
          "111110" when "01100110",   -- pag 3, 06h: a12 = -2
          "000110" when "01100111",   -- pag 3, 07h: a22 = 6
          "000010" when "01101000",   -- pag 3, 08h: a32 = 2
          "111111" when "01101001",   -- pag 3, 09h: a42 = -1
          "000101" when "01101010",   -- pag 3, 0Ah: a52 = 5
          "000001" when "01101011",   -- pag 3, 0Bh: a13 = 1
          "000010" when "01101100",   -- pag 3, 0Ch: a23 = 2
          "111001" when "01101101",   -- pag 3, 0Dh: a33 = -7
          "000100" when "01101110",   -- pag 3, 0Eh: a43 = 4
          "000010" when "01101111",   -- pag 3, 0Fh: a53 = 2
          "000011" when "01110000",   -- pag 3, 10h: a14 = 3
          "111111" when "01110001",   -- pag 3, 11h: a24 = -1
          "000100" when "01110010",   -- pag 3, 12h: a34 = 4
          "000101" when "01110011",   -- pag 3, 13h: a44 = 5
          "111101" when "01110100",   -- pag 3, 14h: a54 = -3
          "000000" when "01110101",   -- pag 3, 15h: a15 = 0
          "000101" when "01110110",   -- pag 3, 16h: a25 = 5
          "000010" when "01110111",   -- pag 3, 17h: a35 = 2
          "111101" when "01111000",   -- pag 3, 18h: a45 = -3
          "001000" when "01111001",   -- pag 3, 19h: a55 = 8
          "000011" when "10000000",   -- pag 4, 00h: n = 3
          "111110" when "10000001",   -- pag 4, 01h: a11 = -2
          "000100" when "10000010",   -- pag 4, 02h: a21 = 4
          "000001" when "10000011",   -- pag 4, 03h: a31 = 1
          "000100" when "10000100",   -- pag 4, 04h: a12 = 4
          "111000" when "10000101",   -- pag 4, 05h: a22 = -8
          "000011" when "10000110",   -- pag 4, 06h: a32 = 3
          "000001" when "10000111",   -- pag 4, 07h: a13 = 1
          "000011" when "10001000",   -- pag 4, 08h: a23 = 3
          "000101" when "10001001",   -- pag 4, 09h: a33 = 5
          "000100" when "10100000",   -- pag 5, 00h: n = 4
          "000001" when "10100001",   -- pag 5, 01h: a11 = 1
          "000010" when "10100010",   -- pag 5, 02h: a21 = 2
          "000011" when "10100011",   -- pag 5, 03h: a31 = 3
          "000000" when "10100100",   -- pag 5, 04h: a41 = 0
          "000010" when "10100101",   -- pag 5, 05h: a12 = 2
          "000100" when "10100110",   -- pag 5, 06h: a22 = 4
          "000001" when "10100111",   -- pag 5, 07h: a32 = 1
          "000101" when "10101000",   -- pag 5, 08h: a42 = 5
          "000011" when "10101001",   -- pag 5, 09h: a13 = 3
          "000001" when "10101010",   -- pag 5, 0Ah: a23 = 1
          "111110" when "10101011",   -- pag 5, 0Bh: a33 = -2
          "000100" when "10101100",   -- pag 5, 0Ch: a43 = 4
          "000000" when "10101101",   -- pag 5, 0Dh: a14 = 0
          "000101" when "10101110",   -- pag 5, 0Eh: a24 = 5
          "000100" when "10101111",   -- pag 5, 0Fh: a34 = 4
          "111101" when "10110000",   -- pag 5, 10h: a44 = -3
          "000101" when "11000000",   -- pag 6, 00h: n = 5
          "000000" when "11000001",   -- pag 6, 01h: a11 = 0
          "000000" when "11000010",   -- pag 6, 02h: a21 = 0
          "000000" when "11000011",   -- pag 6, 03h: a31 = 0
          "000000" when "11000100",   -- pag 6, 04h: a41 = 0
          "000000" when "11000101",   -- pag 6, 05h: a51 = 0
          "000000" when "11000110",   -- pag 6, 06h: a12 = 0
          "000011" when "11000111",   -- pag 6, 07h: a22 = 3
          "000001" when "11001000",   -- pag 6, 08h: a32 = 1
          "000010" when "11001001",   -- pag 6, 09h: a42 = 2
          "000100" when "11001010",   -- pag 6, 0Ah: a52 = 4
          "000000" when "11001011",   -- pag 6, 0Bh: a13 = 0
          "000001" when "11001100",   -- pag 6, 0Ch: a23 = 1
          "000101" when "11001101",   -- pag 6, 0Dh: a33 = 5
          "111110" when "11001110",   -- pag 6, 0Eh: a43 = -2
          "000001" when "11001111",   -- pag 6, 0Fh: a53 = 1
          "000000" when "11010000",   -- pag 6, 10h: a14 = 0
          "000010" when "11010001",   -- pag 6, 11h: a24 = 2
          "111110" when "11010010",   -- pag 6, 12h: a34 = -2
          "000110" when "11010011",   -- pag 6, 13h: a44 = 6
          "000011" when "11010100",   -- pag 6, 14h: a54 = 3
          "000000" when "11010101",   -- pag 6, 15h: a15 = 0
          "000100" when "11010110",   -- pag 6, 16h: a25 = 4
          "000001" when "11010111",   -- pag 6, 17h: a35 = 1
          "000011" when "11011000",   -- pag 6, 18h: a45 = 3
          "000111" when "11011001",   -- pag 6, 19h: a55 = 7
          "000101" when "11100000",   -- pag 7, 00h: n = 5
          "011110" when "11100001",   -- pag 7, 01h: a11 = 30
          "011011" when "11100010",   -- pag 7, 02h: a21 = 27
          "011000" when "11100011",   -- pag 7, 03h: a31 = 24
          "100011" when "11100100",   -- pag 7, 04h: a41 = -29
          "100001" when "11100101",   -- pag 7, 05h: a51 = -31
          "011011" when "11100110",   -- pag 7, 06h: a12 = 27
          "101011" when "11100111",   -- pag 7, 07h: a22 = -21
          "011100" when "11101000",   -- pag 7, 08h: a32 = 28
```

```
          "011101" when "11101001",   -- pag 7, 09h: a42 = 29
          "101000" when "11101010",   -- pag 7, 0Ah: a52 = -24
          "011000" when "11101011",   -- pag 7, 0Bh: a13 = 24
          "011100" when "11101100",   -- pag 7, 0Ch: a23 = 28
          "000010" when "11101101",   -- pag 7, 0Dh: a33 = 2
          "100100" when "11101110",   -- pag 7, 0Eh: a43 = -28
          "100100" when "11101111",   -- pag 7, 0Fh: a53 = -28
          "100011" when "11110000",   -- pag 7, 10h: a14 = -29
          "011101" when "11110001",   -- pag 7, 11h: a24 = 29
          "100100" when "11110010",   -- pag 7, 12h: a34 = -28
          "100101" when "11110011",   -- pag 7, 13h: a44 = -27
          "011100" when "11110100",   -- pag 7, 14h: a54 = 28
          "100001" when "11110101",   -- pag 7, 15h: a15 = -31
          "101000" when "11110110",   -- pag 7, 16h: a25 = -24
          "100100" when "11110111",   -- pag 7, 17h: a35 = -28
          "011100" when "11111000",   -- pag 7, 18h: a45 = 28
          "100010" when "11111001",   -- pag 7, 19h: a55 = -30
          "000000" when others;
end RTL;


--------------------------------------------------------------------------------
-- RAM DE TRABAJO: 64 x 32 bits. Escritura sincrona, lectura asincrona.
-- dir = fila(3) & columna(3).
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity ram_trabajo is
    Port ( clk  : in  STD_LOGIC;
           we   : in  STD_LOGIC;
           dir  : in  STD_LOGIC_VECTOR (5 downto 0);
           din  : in  STD_LOGIC_VECTOR (31 downto 0);
           dout : out STD_LOGIC_VECTOR (31 downto 0));
end ram_trabajo;

architecture RTL of ram_trabajo is
type ram_tipo is array (0 to 63) of std_logic_vector (31 downto 0);
signal ram : ram_tipo := (others => (others => '0'));   -- contenido inicial (INIT)
begin

process (clk)
begin
    if clk'event and clk = '1' then
        if we = '1' then
            ram(conv_integer(dir)) <= din;
        end if;
    end if;
end process;

dout <= ram(conv_integer(dir));

end RTL;


--------------------------------------------------------------------------------
-- MULTIPLICADOR CON SIGNO 25 x 25 -> 50 bits (combinacional).
-- std_logic_signed interpreta los std_logic_vector en complemento a 2.
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_signed.all;

entity multiplicador is
    Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
           b : in  STD_LOGIC_VECTOR (24 downto 0);
           p : out STD_LOGIC_VECTOR (49 downto 0));
end multiplicador;

architecture RTL of multiplicador is
begin
p <= a * b;
end RTL;


--------------------------------------------------------------------------------
-- SUMADOR/RESTADOR DE 50 bits (combinacional).
-- op = '0': s = a + b      op = '1': s = a + not(b) + 1 = a - b
-- Un solo sumador con acarreo de entrada.
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity sumador_restador is
    Port ( a  : in  STD_LOGIC_VECTOR (49 downto 0);
           b  : in  STD_LOGIC_VECTOR (49 downto 0);
```

```
262             op : in  STD_LOGIC;
263             s  : out STD_LOGIC_VECTOR (49 downto 0));
264  end sumador_restador;
265
266  architecture RTL of sumador_restador is
267  signal bx : STD_LOGIC_VECTOR (49 downto 0);
268  begin
269  bx <= not b when op = '1' else b;
270  s  <= a + bx + op;
271  end RTL;
272
273
274  ----------------------------------------------------------------------------------
275  -- CAMINO DE DATOS
276  -- Registros con habilitacion (ld_*, clr_*, inc_*), multiplexores (sel_*),
277  -- memorias y unidades funcionales. Genera las senales de estado para la FSM.
278  ----------------------------------------------------------------------------------
279  library IEEE;
280  use IEEE.STD_LOGIC_1164.ALL;
281  use ieee.std_logic_unsigned.all;
282
283  entity camino_datos is
284      Port ( clk, rst  : in  STD_LOGIC;
285             sw_mat    : in  STD_LOGIC_VECTOR (2 downto 0);
286             sw_piv    : in  STD_LOGIC_VECTOR (2 downto 0);
287             -- senales de control (desde la unidad de control)
288             ld_pag, clr_dira, inc_dira : in STD_LOGIC;
289             we_ram    : in  STD_LOGIC;
290             sel_fila  : in  STD_LOGIC_VECTOR (1 downto 0);
291             sel_col   : in  STD_LOGIC_VECTOR (1 downto 0);
292             sel_din   : in  STD_LOGIC_VECTOR (2 downto 0);
293             ld_n      : in  STD_LOGIC;
294             ld_i      : in  STD_LOGIC;
295             sel_i     : in  STD_LOGIC_VECTOR (1 downto 0);
296             ld_j      : in  STD_LOGIC;
297             sel_j     : in  STD_LOGIC_VECTOR (1 downto 0);
298             ld_k, sel_k            : in STD_LOGIC;
299             ld_piv                 : in STD_LOGIC;
300             ld_p, sel_p            : in STD_LOGIC;
301             ld_pabs   : in  STD_LOGIC;
302             sel_pabs  : in  STD_LOGIC_VECTOR (1 downto 0);
303             clr_signo, tog_signo   : in STD_LOGIC;
304             clr_banco, ld_banco    : in STD_LOGIC;
305             ld_tmp1, ld_tmp2       : in STD_LOGIC;
306             ld_ma, sel_ma, ld_mb   : in STD_LOGIC;
307             ld_t1, ld_t2, ld_t3    : in STD_LOGIC;
308             ld_r, sel_r, clr_q, inc_q, ld_sq : in STD_LOGIC;
309             ld_det    : in  STD_LOGIC;
310             sel_det   : in  STD_LOGIC_VECTOR (1 downto 0);
311             sel_sra   : in  STD_LOGIC_VECTOR (1 downto 0);
312             sel_srb   : in  STD_LOGIC_VECTOR (2 downto 0);
313             op_sr     : in  STD_LOGIC;
314             ld_selant, ld_neg      : in STD_LOGIC;
315             ld_bin    : in  STD_LOGIC;
316             sel_bin   : in  STD_LOGIC_VECTOR (1 downto 0);
317             clr_bcd, ins_bcd, aj_bcd, sh_bcd : in STD_LOGIC;
318             clr_cont, inc_cont     : in STD_LOGIC;
319             clr_d, inc_d           : in STD_LOGIC;
320             ld_disp                : in STD_LOGIC;
321             -- senales de estado (hacia la unidad de control)
322             z_dout, i_eq_n, j_eq_n, k1_eq_n      : out STD_LOGIC;
323             t3_neg, r_ge, sq, piv_neg            : out STD_LOGIC;
324             signo, val_neg                       : out STD_LOGIC;
325             cont_31, dig_gt4, d_9, sel_dif       : out STD_LOGIC;
326             -- salida hacia el display
327             disp_bcd  : out STD_LOGIC_VECTOR (27 downto 0);
328             disp_neg  : out STD_LOGIC;
329             desborde  : out STD_LOGIC);
330  end camino_datos;
331
332  architecture RTL of camino_datos is
333
334  component rom_matrices
335      Port ( dir : in  STD_LOGIC_VECTOR (7 downto 0);
336             a   : out STD_LOGIC_VECTOR (5 downto 0));
337  end component;
338
339  component ram_trabajo
340      Port ( clk  : in  STD_LOGIC;
341             we   : in  STD_LOGIC;
342             dir  : in  STD_LOGIC_VECTOR (5 downto 0);
343             din  : in  STD_LOGIC_VECTOR (31 downto 0);
344             dout : out STD_LOGIC_VECTOR (31 downto 0));
345  end component;
346
347  component multiplicador
348      Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
349             b : in  STD_LOGIC_VECTOR (24 downto 0);
```

```
           p : out STD_LOGIC_VECTOR (49 downto 0));
end component;

component sumador_restador
    Port ( a  : in  STD_LOGIC_VECTOR (49 downto 0);
           b  : in  STD_LOGIC_VECTOR (49 downto 0);
           op : in  STD_LOGIC;
           s  : out STD_LOGIC_VECTOR (49 downto 0));
end component;

-- Constantes
constant UNO3   : STD_LOGIC_VECTOR (2 downto 0)  := "001";
constant CERO12 : STD_LOGIC_VECTOR (11 downto 0) := x"000";
constant CERO18 : STD_LOGIC_VECTOR (17 downto 0) := "000000000000000000";
constant CERO32 : STD_LOGIC_VECTOR (31 downto 0) := x"00000000";
constant UNO32  : STD_LOGIC_VECTOR (31 downto 0) := x"00000001";
constant CERO40 : STD_LOGIC_VECTOR (39 downto 0) := x"0000000000";
constant CERO46 : STD_LOGIC_VECTOR (45 downto 0) := (others => '0');
constant CERO50 : STD_LOGIC_VECTOR (49 downto 0) := (others => '0');
constant TRES50 : STD_LOGIC_VECTOR (49 downto 0) := (1 downto 0 => '1', others => '0');

-- Registros
signal pag      : STD_LOGIC_VECTOR (2 downto 0);   -- pagina de la ROM
signal dira     : STD_LOGIC_VECTOR (4 downto 0);   -- direccion de la ROM
signal n, i, j, k : STD_LOGIC_VECTOR (2 downto 0); -- tamano y contadores
signal piv      : STD_LOGIC_VECTOR (31 downto 0);  -- pivote actual a(k,k)
signal p        : STD_LOGIC_VECTOR (31 downto 0);  -- pivote anterior (divisor)
signal pabs     : STD_LOGIC_VECTOR (31 downto 0);  -- |p|
signal signo_r  : STD_LOGIC;                       -- paridad de intercambios
signal b1, b2, b3, b4 : STD_LOGIC_VECTOR (31 downto 0); -- banco de pivotes
signal tmp1, tmp2 : STD_LOGIC_VECTOR (31 downto 0); -- intercambio de filas
signal mul_a, mul_b : STD_LOGIC_VECTOR (24 downto 0); -- entradas del multiplicador
signal t1, t2, t3 : STD_LOGIC_VECTOR (49 downto 0); -- productos y numerador
signal R        : STD_LOGIC_VECTOR (49 downto 0);  -- residuo
signal Q        : STD_LOGIC_VECTOR (31 downto 0);  -- cociente
signal sq_r     : STD_LOGIC;                       -- signo del cociente
signal det      : STD_LOGIC_VECTOR (31 downto 0);  -- determinante
signal sel_ant  : STD_LOGIC_VECTOR (2 downto 0);   -- seleccion ya convertida
signal neg      : STD_LOGIC;                       -- signo del valor convertido
signal bin      : STD_LOGIC_VECTOR (31 downto 0);  -- registro binario (XS3)
signal bcd      : STD_LOGIC_VECTOR (39 downto 0);  -- registro decimal (XS3), 10 digitos
signal cont     : STD_LOGIC_VECTOR (4 downto 0);   -- bits procesados (0..31)
signal d        : STD_LOGIC_VECTOR (3 downto 0);   -- digito revisado (0..9)
signal disp     : STD_LOGIC_VECTOR (39 downto 0);  -- valor que se muestra
signal disp_neg_r : STD_LOGIC;

-- Senales combinacionales
signal dir_rom  : STD_LOGIC_VECTOR (7 downto 0);
signal a        : STD_LOGIC_VECTOR (5 downto 0);
signal ext_a    : STD_LOGIC_VECTOR (25 downto 0);
signal a32      : STD_LOGIC_VECTOR (31 downto 0);
signal fila, col : STD_LOGIC_VECTOR (2 downto 0);
signal dir_ram  : STD_LOGIC_VECTOR (5 downto 0);
signal din_ram  : STD_LOGIC_VECTOR (31 downto 0);
signal dout     : STD_LOGIC_VECTOR (31 downto 0);
signal mul_p    : STD_LOGIC_VECTOR (49 downto 0);
signal sr_a, sr_b, sr_s : STD_LOGIC_VECTOR (49 downto 0);
signal sx_piv, sx_dout, sx_val : STD_LOGIC_VECTOR (17 downto 0);
signal i_mas1, j_mas1, k_mas1 : STD_LOGIC_VECTOR (2 downto 0);
signal i_d, j_d, k_d : STD_LOGIC_VECTOR (2 downto 0);
signal p_d, pabs_d, det_d, bin_d : STD_LOGIC_VECTOR (31 downto 0);
signal ma_d     : STD_LOGIC_VECTOR (24 downto 0);
signal r_d      : STD_LOGIC_VECTOR (49 downto 0);
signal we_b1, we_b2, we_b3, we_b4 : STD_LOGIC;
signal val      : STD_LOGIC_VECTOR (31 downto 0);
signal dig      : STD_LOGIC_VECTOR (3 downto 0);
signal bcd_ins, bcd_aj, bcd_sh : STD_LOGIC_VECTOR (39 downto 0);

begin

-- ------------------------------------------------------------------------
-- Memorias y unidades funcionales
-- ------------------------------------------------------------------------
dir_rom <= pag & dira;
U_ROM : rom_matrices     port map (dir => dir_rom, a => a);
U_RAM : ram_trabajo      port map (clk => clk, we => we_ram, dir => dir_ram,
                                   din => din_ram, dout => dout);
U_MUL : multiplicador    port map (a => mul_a, b => mul_b, p => mul_p);
U_SR  : sumador_restador port map (a => sr_a, b => sr_b, op => op_sr, s => sr_s);

-- ------------------------------------------------------------------------
-- Extensiones de signo
-- ------------------------------------------------------------------------
ext_a   <= (others => a(5));
a32     <= ext_a & a;                       -- dato de la ROM a 32 bits
sx_piv  <= (others => piv(31));
sx_dout <= (others => dout(31));
sx_val  <= (others => val(31));
```

```
438
439  ---------------------------------------------------------------------------
440  -- Direccion y dato de la RAM
441  ---------------------------------------------------------------------------
442  with sel_fila select
443      fila <= i when "00",
444              k when "01",
445              n when others;
446
447  with sel_col select
448      col <= j when "00",
449             k when "01",
450             n when others;
451
452  dir_ram <= fila & col;
453
454  with sel_din select
455      din_ram <= a32               when "000",   -- copia de la ROM
456                 tmp1              when "001",   -- intercambio
457                 tmp2              when "010",   -- intercambio
458                 Q                 when "011",   -- +Q
459                 sr_s(31 downto 0) when others;  -- -Q
460
461  ---------------------------------------------------------------------------
462  -- Operandos del sumador/restador
463  ---------------------------------------------------------------------------
464  with sel_sra select
465      sr_a <= CERO50       when "00",
466              t1           when "01",
467              R            when "10",
468              CERO46 & dig when others;
469
470  with sel_srb select
471      sr_b <= t2              when "000",
472              t3              when "001",
473              CERO18 & pabs   when "010",
474              CERO18 & Q      when "011",
475              sx_piv & piv    when "100",
476              sx_dout & dout  when "101",
477              sx_val & val    when "110",
478              TRES50          when others;
479
480  ---------------------------------------------------------------------------
481  -- Multiplexores de entrada de los registros
482  ---------------------------------------------------------------------------
483  i_mas1 <= i + '1';
484  j_mas1 <= j + '1';
485  k_mas1 <= k + '1';
486
487  with sel_i select
488      i_d <= UNO3   when "00",
489             i_mas1 when "01",
490             k_mas1 when others;
491
492  with sel_j select
493      j_d <= UNO3   when "00",
494             j_mas1 when "01",
495             k_mas1 when "10",
496             k      when others;
497
498  k_d  <= UNO3 when sel_k = '0' else k_mas1;
499  p_d  <= UNO32 when sel_p = '0' else piv;
500  ma_d <= piv(24 downto 0) when sel_ma = '0' else dout(24 downto 0);
501  r_d  <= t3 when sel_r = '0' else sr_s;
502
503  with sel_pabs select
504      pabs_d <= UNO32             when "00",
505                piv               when "01",
506                sr_s(31 downto 0) when others;
507
508  with sel_det select
509      det_d <= CERO32            when "00",
510               dout              when "01",
511               sr_s(31 downto 0) when others;
512
513  with sel_bin select
514      bin_d <= val                     when "00",
515               sr_s(31 downto 0)       when "01",
516               bin(30 downto 0) & '0' when others;   -- corrimiento (XS3 paso 4)
517
518  -- Banco de pivotes: escritura decodificada por k, lectura por sw_piv
519  we_b1 <= '1' when (ld_banco = '1' and k = "001") else '0';
520  we_b2 <= '1' when (ld_banco = '1' and k = "010") else '0';
521  we_b3 <= '1' when (ld_banco = '1' and k = "011") else '0';
522  we_b4 <= '1' when (ld_banco = '1' and k = "100") else '0';
523
524  with sw_piv select
525      val <= det    when "000",
```

```
            b1     when "001",
            b2     when "010",
            b3     when "011",
            b4     when "100",
            CERO32 when others;

-- XS3: digito d del registro decimal
with d select
    dig <= bcd(3 downto 0)   when "0000",
           bcd(7 downto 4)   when "0001",
           bcd(11 downto 8)  when "0010",
           bcd(15 downto 12) when "0011",
           bcd(19 downto 16) when "0100",
           bcd(23 downto 20) when "0101",
           bcd(27 downto 24) when "0110",
           bcd(31 downto 28) when "0111",
           bcd(35 downto 32) when "1000",
           bcd(39 downto 36) when "1001",
           "0000"            when others;

-- XS3 paso 2: el MSB del binario entra al LSB del decimal
bcd_ins <= bcd(39 downto 1) & bin(31);

-- XS3 paso 3: el digito d se reemplaza por digito + 3 (salida del sumador)
bcd_aj(3 downto 0)   <= sr_s(3 downto 0) when d = "0000" else bcd(3 downto 0);
bcd_aj(7 downto 4)   <= sr_s(3 downto 0) when d = "0001" else bcd(7 downto 4);
bcd_aj(11 downto 8)  <= sr_s(3 downto 0) when d = "0010" else bcd(11 downto 8);
bcd_aj(15 downto 12) <= sr_s(3 downto 0) when d = "0011" else bcd(15 downto 12);
bcd_aj(19 downto 16) <= sr_s(3 downto 0) when d = "0100" else bcd(19 downto 16);
bcd_aj(23 downto 20) <= sr_s(3 downto 0) when d = "0101" else bcd(23 downto 20);
bcd_aj(27 downto 24) <= sr_s(3 downto 0) when d = "0110" else bcd(27 downto 24);
bcd_aj(31 downto 28) <= sr_s(3 downto 0) when d = "0111" else bcd(31 downto 28);
bcd_aj(35 downto 32) <= sr_s(3 downto 0) when d = "1000" else bcd(35 downto 32);
bcd_aj(39 downto 36) <= sr_s(3 downto 0) when d = "1001" else bcd(39 downto 36);

-- XS3 paso 4: corrimiento a la izquierda, entra 0
bcd_sh <= bcd(38 downto 0) & '0';

--------------------------------------------------------------------------------
-- Registros
--------------------------------------------------------------------------------
process (clk, rst)
begin
if rst = '1' then
    pag  <= "000";   dira <= "00000";
    n    <= "000";   i    <= "000";   j <= "000";   k <= "000";
    piv  <= CERO32;  p    <= CERO32;  pabs <= CERO32;  signo_r <= '0';
    b1   <= CERO32;  b2   <= CERO32;  b3   <= CERO32;  b4 <= CERO32;
    tmp1 <= CERO32;  tmp2 <= CERO32;
    mul_a <= (others => '0');  mul_b <= (others => '0');
    t1   <= CERO50;  t2   <= CERO50;  t3   <= CERO50;
    R    <= CERO50;  Q    <= CERO32;  sq_r <= '0';
    det  <= CERO32;  sel_ant <= "000"; neg <= '0';
    bin  <= CERO32;  bcd  <= CERO40;  cont <= "00000";  d <= "0000";
    disp <= CERO40;  disp_neg_r <= '0';

elsif clk'event and clk = '1' then

    -- Carga de la matriz
    if ld_pag = '1' then pag <= sw_mat; end if;

    if clr_dira = '1' then
        dira <= "00000";
    elsif inc_dira = '1' then
        dira <= dira + '1';
    end if;

    if ld_n = '1' then n <= a(2 downto 0); end if;
    if ld_i = '1' then i <= i_d; end if;
    if ld_j = '1' then j <= j_d; end if;
    if ld_k = '1' then k <= k_d; end if;

    -- Bareiss
    if ld_piv  = '1' then piv  <= dout;   end if;
    if ld_p    = '1' then p    <= p_d;    end if;
    if ld_pabs = '1' then pabs <= pabs_d; end if;

    if clr_signo = '1' then
        signo_r <= '0';
    elsif tog_signo = '1' then
        signo_r <= not signo_r;
    end if;

    if clr_banco = '1' then
        b1 <= CERO32;  b2 <= CERO32;  b3 <= CERO32;  b4 <= CERO32;
    else
        if we_b1 = '1' then b1 <= piv; end if;
        if we_b2 = '1' then b2 <= piv; end if;
```

```
614          if we_b3 = '1' then b3 <= piv; end if;
615          if we_b4 = '1' then b4 <= piv; end if;
616      end if;
617
618      if ld_tmp1 = '1' then tmp1 <= dout; end if;
619      if ld_tmp2 = '1' then tmp2 <= dout; end if;
620
621      if ld_ma = '1' then mul_a <= ma_d;             end if;
622      if ld_mb = '1' then mul_b <= dout(24 downto 0); end if;
623      if ld_t1 = '1' then t1 <= mul_p; end if;
624      if ld_t2 = '1' then t2 <= mul_p; end if;
625      if ld_t3 = '1' then t3 <= sr_s;  end if;
626
627      -- Division por restas sucesivas
628      if ld_r  = '1' then R    <= r_d;               end if;
629      if ld_sq = '1' then sq_r <= t3(49) xor p(31);  end if;
630
631      if clr_q = '1' then
632          Q <= CERO32;
633      elsif inc_q = '1' then
634          Q <= Q + '1';
635      end if;
636
637      if ld_det = '1' then det <= det_d; end if;
638
639      -- XS3 y visualizacion
640      if ld_selant = '1' then sel_ant <= sw_piv;  end if;
641      if ld_neg    = '1' then neg     <= val(31); end if;
642      if ld_bin    = '1' then bin     <= bin_d;   end if;
643
644      if clr_bcd = '1' then
645          bcd <= CERO40;                -- paso 1
646      elsif ins_bcd = '1' then
647          bcd <= bcd_ins;               -- paso 2
648      elsif aj_bcd = '1' then
649          bcd <= bcd_aj;                -- paso 3
650      elsif sh_bcd = '1' then
651          bcd <= bcd_sh;                -- paso 4
652      end if;
653
654      if clr_cont = '1' then
655          cont <= "00000";
656      elsif inc_cont = '1' then
657          cont <= cont + '1';
658      end if;
659
660      if clr_d = '1' then
661          d <= "0000";
662      elsif inc_d = '1' then
663          d <= d + '1';
664      end if;
665
666      if ld_disp = '1' then
667          disp       <= bcd;
668          disp_neg_r <= neg;
669      end if;
670
671  end if;
672  end process;
673
674  ------------------------------------------------------------------------------
675  -- Senales de estado
676  ------------------------------------------------------------------------------
677  z_dout  <= '1' when dout = CERO32 else '0';      -- M(.,.) = 0
678  i_eq_n  <= '1' when i = n else '0';
679  j_eq_n  <= '1' when j = n else '0';
680  k1_eq_n <= '1' when k_mas1 = n else '0';         -- k = n - 1
681  t3_neg  <= t3(49);
682  r_ge    <= not sr_s(49);                         -- R - |p| >= 0
683  sq      <= sq_r;
684  piv_neg <= piv(31);
685  signo   <= signo_r;
686  val_neg <= val(31);
687  cont_31 <= '1' when cont = "11111" else '0';
688  dig_gt4 <= '1' when dig > "0100" else '0';       -- digito >= 5
689  d_9     <= '1' when d = "1001" else '0';
690  sel_dif <= '1' when sw_piv /= sel_ant else '0';
691
692  ------------------------------------------------------------------------------
693  -- Salidas hacia el display
694  ------------------------------------------------------------------------------
695  disp_bcd <= disp(27 downto 0);                   -- 7 digitos
696  disp_neg <= disp_neg_r;
697  desborde <= '0' when disp(39 downto 28) = CERO12 else '1';
698
699  end RTL;
700
701
```

```
702  ---------------------------------------------------------------------------------
703  -- UNIDAD DE CONTROL: FSM del diagrama ASM (27 estados, S0 a S26).
704  -- Proceso 1: registro de estado.  Proceso 2: logica combinacional de estado
705  -- siguiente y de salidas (Moore en las cajas de estado, Mealy en las cajas
706  -- de salida condicional).
707  ---------------------------------------------------------------------------------
708  library IEEE;
709  use IEEE.STD_LOGIC_1164.ALL;
710
711  entity unidad_control is
712      Port ( clk, rst  : in  STD_LOGIC;
713             inicio    : in  STD_LOGIC;
714             -- senales de estado (desde el camino de datos)
715             z_dout, i_eq_n, j_eq_n, k1_eq_n      : in STD_LOGIC;
716             t3_neg, r_ge, sq, piv_neg            : in STD_LOGIC;
717             signo, val_neg                       : in STD_LOGIC;
718             cont_31, dig_gt4, d_9, sel_dif       : in STD_LOGIC;
719             -- senales de control (hacia el camino de datos)
720             ld_pag, clr_dira, inc_dira : out STD_LOGIC;
721             we_ram    : out STD_LOGIC;
722             sel_fila  : out STD_LOGIC_VECTOR (1 downto 0);
723             sel_col   : out STD_LOGIC_VECTOR (1 downto 0);
724             sel_din   : out STD_LOGIC_VECTOR (2 downto 0);
725             ld_n      : out STD_LOGIC;
726             ld_i      : out STD_LOGIC;
727             sel_i     : out STD_LOGIC_VECTOR (1 downto 0);
728             ld_j      : out STD_LOGIC;
729             sel_j     : out STD_LOGIC_VECTOR (1 downto 0);
730             ld_k, sel_k            : out STD_LOGIC;
731             ld_piv                 : out STD_LOGIC;
732             ld_p, sel_p            : out STD_LOGIC;
733             ld_pabs   : out STD_LOGIC;
734             sel_pabs  : out STD_LOGIC_VECTOR (1 downto 0);
735             clr_signo, tog_signo   : out STD_LOGIC;
736             clr_banco, ld_banco    : out STD_LOGIC;
737             ld_tmp1, ld_tmp2       : out STD_LOGIC;
738             ld_ma, sel_ma, ld_mb   : out STD_LOGIC;
739             ld_t1, ld_t2, ld_t3    : out STD_LOGIC;
740             ld_r, sel_r, clr_q, inc_q, ld_sq : out STD_LOGIC;
741             ld_det    : out STD_LOGIC;
742             sel_det   : out STD_LOGIC_VECTOR (1 downto 0);
743             sel_sra   : out STD_LOGIC_VECTOR (1 downto 0);
744             sel_srb   : out STD_LOGIC_VECTOR (2 downto 0);
745             op_sr     : out STD_LOGIC;
746             ld_selant, ld_neg      : out STD_LOGIC;
747             ld_bin    : out STD_LOGIC;
748             sel_bin   : out STD_LOGIC_VECTOR (1 downto 0);
749             clr_bcd, ins_bcd, aj_bcd, sh_bcd : out STD_LOGIC;
750             clr_cont, inc_cont     : out STD_LOGIC;
751             clr_d, inc_d           : out STD_LOGIC;
752             ld_disp                : out STD_LOGIC;
753             final                  : out STD_LOGIC);
754  end unidad_control;
755
756  architecture RTL of unidad_control is
757
758  -- Codificacion de los estados
759  constant S0  : STD_LOGIC_VECTOR (4 downto 0) := "00000";
760  constant S1  : STD_LOGIC_VECTOR (4 downto 0) := "00001";
761  constant S2  : STD_LOGIC_VECTOR (4 downto 0) := "00010";
762  constant S3  : STD_LOGIC_VECTOR (4 downto 0) := "00011";
763  constant S4  : STD_LOGIC_VECTOR (4 downto 0) := "00100";
764  constant S5  : STD_LOGIC_VECTOR (4 downto 0) := "00101";
765  constant S6  : STD_LOGIC_VECTOR (4 downto 0) := "00110";
766  constant S7  : STD_LOGIC_VECTOR (4 downto 0) := "00111";
767  constant S8  : STD_LOGIC_VECTOR (4 downto 0) := "01000";
768  constant S9  : STD_LOGIC_VECTOR (4 downto 0) := "01001";
769  constant S10 : STD_LOGIC_VECTOR (4 downto 0) := "01010";
770  constant S11 : STD_LOGIC_VECTOR (4 downto 0) := "01011";
771  constant S12 : STD_LOGIC_VECTOR (4 downto 0) := "01100";
772  constant S13 : STD_LOGIC_VECTOR (4 downto 0) := "01101";
773  constant S14 : STD_LOGIC_VECTOR (4 downto 0) := "01110";
774  constant S15 : STD_LOGIC_VECTOR (4 downto 0) := "01111";
775  constant S16 : STD_LOGIC_VECTOR (4 downto 0) := "10000";
776  constant S17 : STD_LOGIC_VECTOR (4 downto 0) := "10001";
777  constant S18 : STD_LOGIC_VECTOR (4 downto 0) := "10010";
778  constant S19 : STD_LOGIC_VECTOR (4 downto 0) := "10011";
779  constant S20 : STD_LOGIC_VECTOR (4 downto 0) := "10100";
780  constant S21 : STD_LOGIC_VECTOR (4 downto 0) := "10101";
781  constant S22 : STD_LOGIC_VECTOR (4 downto 0) := "10110";
782  constant S23 : STD_LOGIC_VECTOR (4 downto 0) := "10111";
783  constant S24 : STD_LOGIC_VECTOR (4 downto 0) := "11000";
784  constant S25 : STD_LOGIC_VECTOR (4 downto 0) := "11001";
785  constant S26 : STD_LOGIC_VECTOR (4 downto 0) := "11010";
786
787  signal st, prox : STD_LOGIC_VECTOR (4 downto 0);
788
789  begin
```

```
790
791  -- Registro de estado
792  process (clk, rst)
793  begin
794      if rst = '1' then
795          st <= S0;
796      elsif clk'event and clk = '1' then
797          st <= prox;
798      end if;
799  end process;
800
801  -- Logica de estado siguiente y de salidas
802  process (st, inicio, z_dout, i_eq_n, j_eq_n, k1_eq_n, t3_neg, r_ge, sq,
803           piv_neg, signo, val_neg, cont_31, dig_gt4, d_9, sel_dif)
804  begin
805      -- Valores por defecto: ninguna operacion, se mantiene el estado
806      prox      <= st;
807      ld_pag    <= '0';  clr_dira  <= '0';  inc_dira  <= '0';
808      we_ram    <= '0';  sel_fila  <= "00"; sel_col   <= "00"; sel_din <= "000";
809      ld_n      <= '0';
810      ld_i      <= '0';  sel_i     <= "00";
811      ld_j      <= '0';  sel_j     <= "00";
812      ld_k      <= '0';  sel_k     <= '0';
813      ld_piv    <= '0';
814      ld_p      <= '0';  sel_p     <= '0';
815      ld_pabs   <= '0';  sel_pabs  <= "00";
816      clr_signo <= '0';  tog_signo <= '0';
817      clr_banco <= '0';  ld_banco  <= '0';
818      ld_tmp1   <= '0';  ld_tmp2   <= '0';
819      ld_ma     <= '0';  sel_ma    <= '0';  ld_mb     <= '0';
820      ld_t1     <= '0';  ld_t2     <= '0';  ld_t3     <= '0';
821      ld_r      <= '0';  sel_r     <= '0';  clr_q     <= '0';  inc_q <= '0';  ld_sq <= '0';
822      ld_det    <= '0';  sel_det   <= "00";
823      sel_sra   <= "00"; sel_srb   <= "000"; op_sr    <= '1';
824      ld_selant <= '0';  ld_neg    <= '0';
825      ld_bin    <= '0';  sel_bin   <= "00";
826      clr_bcd   <= '0';  ins_bcd   <= '0';  aj_bcd    <= '0';  sh_bcd <= '0';
827      clr_cont  <= '0';  inc_cont  <= '0';
828      clr_d     <= '0';  inc_d     <= '0';
829      ld_disp   <= '0';
830      final     <= '0';
831
832      case st is
833
834          ---------------------------------------------------------------
835          -- Espera y carga de la matriz
836          ---------------------------------------------------------------
837          when S0 =>                              -- final = 0 ; dira <- 0
838              clr_dira <= '1';
839              if inicio = '1' then
840                  ld_pag <= '1';                  -- pag <- sw_mat
841                  prox   <= S1;
842              else
843                  prox   <= S0;
844              end if;
845
846          when S1 =>                              -- n <- ROM(00h) ; dira <- 1 ; i <- 1 ; j <- 1
847              ld_n     <= '1';
848              inc_dira <= '1';
849              ld_i     <= '1';  sel_i <= "00";
850              ld_j     <= '1';  sel_j <= "00";
851              prox     <= S2;
852
853          when S2 =>                              -- M(i,j) <- ROM(dira) ; dira <- dira + 1
854              we_ram   <= '1';  sel_fila <= "00"; sel_col <= "00"; sel_din <= "000";
855              inc_dira <= '1';
856              if i_eq_n = '1' then
857                  ld_i <= '1';  sel_i <= "00";    -- i <- 1
858                  if j_eq_n = '1' then
859                      prox <= S3;
860                  else
861                      ld_j <= '1';  sel_j <= "01";-- j <- j + 1
862                      prox <= S2;
863                  end if;
864              else
865                  ld_i <= '1';  sel_i <= "01";    -- i <- i + 1
866                  prox <= S2;
867              end if;
868
869          when S3 =>                              -- k <- 1 ; p <- 1 ; |p| <- 1 ; signo <- 0 ; banco <- 0
870              ld_k      <= '1';  sel_k    <= '0';
871              ld_p      <= '1';  sel_p    <= '0';
872              ld_pabs   <= '1';  sel_pabs <= "00";
873              clr_signo <= '1';
874              clr_banco <= '1';
875              prox      <= S4;
876
877          ---------------------------------------------------------------
```

```
        -- Pivote, busqueda de fila e intercambio
        ------------------------------------------------------------------
        when S4 =>                              -- piv <- M(k,k)
            sel_fila <= "01";  sel_col <= "01";
            ld_piv   <= '1';
            if z_dout = '1' then
                ld_i <= '1';  sel_i <= "10";    -- i <- k + 1
                prox <= S5;
            else
                prox <= S10;
            end if;

        when S5 =>                              -- lee M(i,k)
            sel_fila <= "00";  sel_col <= "01";
            if z_dout = '0' then
                ld_j      <= '1';  sel_j <= "11";   -- j <- k
                tog_signo <= '1';                   -- signo <- not signo
                prox      <= S6;
            elsif i_eq_n = '1' then
                ld_det <= '1';  sel_det <= "00";    -- det <- 0 (singular)
                prox   <= S21;
            else
                ld_i <= '1';  sel_i <= "01";        -- i <- i + 1
                prox <= S5;
            end if;

        when S6 =>                              -- tmp1 <- M(k,j)
            sel_fila <= "01";  sel_col <= "00";
            ld_tmp1  <= '1';
            prox     <= S7;

        when S7 =>                              -- tmp2 <- M(i,j)
            sel_fila <= "00";  sel_col <= "00";
            ld_tmp2  <= '1';
            prox     <= S8;

        when S8 =>                              -- M(k,j) <- tmp2
            we_ram   <= '1';  sel_fila <= "01"; sel_col <= "00"; sel_din <= "010";
            prox     <= S9;

        when S9 =>                              -- M(i,j) <- tmp1
            we_ram   <= '1';  sel_fila <= "00"; sel_col <= "00"; sel_din <= "001";
            if j_eq_n = '1' then
                prox <= S4;
            else
                ld_j <= '1';  sel_j <= "01";    -- j <- j + 1
                prox <= S6;
            end if;

        when S10 =>                             -- banco(k) <- piv ; i <- k + 1 ; j <- k + 1
            ld_banco <= '1';
            ld_i     <= '1';  sel_i <= "10";
            ld_j     <= '1';  sel_j <= "10";
            prox     <= S11;

        ------------------------------------------------------------------
        -- a(i,j) = ( a(k,k)*a(i,j) - a(i,k)*a(k,j) ) / p
        ------------------------------------------------------------------
        when S11 =>                             -- mul_a <- piv ; mul_b <- M(i,j)
            sel_fila <= "00";  sel_col <= "00";
            ld_ma    <= '1';  sel_ma <= '0';
            ld_mb    <= '1';
            prox     <= S12;

        when S12 =>                             -- t1 <- mul_a x mul_b ; mul_a <- M(i,k)
            sel_fila <= "00";  sel_col <= "01";
            ld_t1    <= '1';
            ld_ma    <= '1';  sel_ma <= '1';
            prox     <= S13;

        when S13 =>                             -- mul_b <- M(k,j)
            sel_fila <= "01";  sel_col <= "00";
            ld_mb    <= '1';
            prox     <= S14;

        when S14 =>                             -- t2 <- mul_a x mul_b
            ld_t2    <= '1';
            prox     <= S15;

        when S15 =>                             -- t3 <- t1 - t2
            sel_sra  <= "01";  sel_srb <= "000";  op_sr <= '1';
            ld_t3    <= '1';
            prox     <= S16;

        when S16 =>                             -- Q <- 0 ; sq <- signo(t3) xor signo(p) ; R <- |t3|
            sel_sra  <= "00";  sel_srb <= "001";  op_sr <= '1';   -- 0 - t3
            clr_q    <= '1';
            ld_sq    <= '1';
```

```
            ld_r     <= '1';
            if t3_neg = '1' then
                sel_r <= '1';                   -- R <- 0 - t3
            else
                sel_r <= '0';                   -- R <- t3
            end if;
            prox     <= S17;

        when S17 =>                             -- division: R - |p|
            sel_sra  <= "10";  sel_srb <= "010";  op_sr <= '1';
            if r_ge = '1' then
                ld_r  <= '1';  sel_r <= '1';    -- R <- R - |p|
                inc_q <= '1';                   -- Q <- Q + 1
                prox  <= S17;
            else
                prox  <= S18;
            end if;

        when S18 =>                             -- M(i,j) <- Q o 0 - Q
            sel_sra  <= "00";  sel_srb <= "011";  op_sr <= '1';   -- 0 - Q
            we_ram   <= '1';  sel_fila <= "00"; sel_col <= "00";
            if sq = '1' then
                sel_din <= "100";               -- M(i,j) <- 0 - Q
            else
                sel_din <= "011";               -- M(i,j) <- Q
            end if;
            if j_eq_n = '1' then
                if i_eq_n = '1' then
                    prox <= S19;
                else
                    ld_i <= '1';  sel_i <= "01";-- i <- i + 1
                    ld_j <= '1';  sel_j <= "10";-- j <- k + 1
                    prox <= S11;
                end if;
            else
                ld_j <= '1';  sel_j <= "01";    -- j <- j + 1
                prox <= S11;
            end if;

        when S19 =>                             -- p <- piv ; |p| <- |piv| ; k <- k + 1
            sel_sra  <= "00";  sel_srb <= "100";  op_sr <= '1';   -- 0 - piv
            ld_p     <= '1';  sel_p <= '1';
            ld_pabs  <= '1';
            if piv_neg = '1' then
                sel_pabs <= "10";               -- |p| <- 0 - piv
            else
                sel_pabs <= "01";               -- |p| <- piv
            end if;
            ld_k     <= '1';  sel_k <= '1';
            if k1_eq_n = '1' then
                prox <= S20;
            else
                prox <= S4;
            end if;

        when S20 =>                             -- det <- M(n,n) o 0 - M(n,n)
            sel_fila <= "10";  sel_col <= "10";
            sel_sra  <= "00";  sel_srb <= "101";  op_sr <= '1';   -- 0 - M(n,n)
            ld_det   <= '1';
            if signo = '1' then
                sel_det <= "10";
            else
                sel_det <= "01";
            end if;
            prox     <= S21;

        ------------------------------------------------------------------
        -- Valor a mostrar y algoritmo XS3
        ------------------------------------------------------------------
        when S21 =>                             -- bin <- |val| ; neg <- signo(val) ; sel_ant <- sw_piv
            sel_sra   <= "00";  sel_srb <= "110";  op_sr <= '1';  -- 0 - val
            ld_selant <= '1';
            ld_neg    <= '1';
            ld_bin    <= '1';
            if val_neg = '1' then
                sel_bin <= "01";                -- bin <- 0 - val
            else
                sel_bin <= "00";                -- bin <- val
            end if;
            prox      <= S22;

        when S22 =>                             -- XS3 paso 1: BCD <- 0 ; cont <- 0
            clr_bcd  <= '1';
            clr_cont <= '1';
            prox     <= S23;

        when S23 =>                             -- XS3 paso 2: BCD(0) <- bin(31)
            ins_bcd  <= '1';
```

```
            if cont_31 = '1' then
                prox <= S26;                    -- paso 5: ultimo paso 2
            else
                clr_d <= '1';                   -- d <- 0
                prox  <= S24;
            end if;

        when S24 =>                             -- XS3 paso 3: si BCD(d) > 4, BCD(d) <- BCD(d) + 3
            sel_sra  <= "11";  sel_srb <= "111";  op_sr <= '0';   -- digito + 3
            if dig_gt4 = '1' then
                aj_bcd <= '1';
            end if;
            if d_9 = '1' then
                prox <= S25;
            else
                inc_d <= '1';                   -- d <- d + 1
                prox  <= S24;
            end if;

        when S25 =>                             -- XS3 paso 4: BCD << 1 ; bin << 1 ; cont <- cont + 1
            sh_bcd   <= '1';
            ld_bin   <= '1';  sel_bin <= "10";
            inc_cont <= '1';
            prox     <= S23;

        when S26 =>                             -- final = 1 ; display <- BCD ; dira <- 0
            final    <= '1';
            ld_disp  <= '1';
            clr_dira <= '1';
            if inicio = '1' then
                ld_pag <= '1';                  -- pag <- sw_mat
                prox   <= S1;
            elsif sel_dif = '1' then
                prox   <= S21;                  -- cambiaron los switches
            else
                prox   <= S26;
            end if;

        when others =>
            prox <= S0;

    end case;
end process;

end RTL;


----------------------------------------------------------------------------------
-- PROCESADOR: unidad de control + camino de datos (descripcion estructural)
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity procesador is
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);
           sw_piv   : in  STD_LOGIC_VECTOR (2 downto 0);
           final    : out STD_LOGIC;
           disp_bcd : out STD_LOGIC_VECTOR (27 downto 0);
           disp_neg : out STD_LOGIC;
           desborde : out STD_LOGIC);
end procesador;

architecture Estructural of procesador is

component unidad_control
    Port ( clk, rst  : in  STD_LOGIC;
           inicio    : in  STD_LOGIC;
           z_dout, i_eq_n, j_eq_n, k1_eq_n      : in STD_LOGIC;
           t3_neg, r_ge, sq, piv_neg            : in STD_LOGIC;
           signo, val_neg                       : in STD_LOGIC;
           cont_31, dig_gt4, d_9, sel_dif       : in STD_LOGIC;
           ld_pag, clr_dira, inc_dira : out STD_LOGIC;
           we_ram    : out STD_LOGIC;
           sel_fila  : out STD_LOGIC_VECTOR (1 downto 0);
           sel_col   : out STD_LOGIC_VECTOR (1 downto 0);
           sel_din   : out STD_LOGIC_VECTOR (2 downto 0);
           ld_n      : out STD_LOGIC;
           ld_i      : out STD_LOGIC;
           sel_i     : out STD_LOGIC_VECTOR (1 downto 0);
           ld_j      : out STD_LOGIC;
           sel_j     : out STD_LOGIC_VECTOR (1 downto 0);
           ld_k, sel_k           : out STD_LOGIC;
           ld_piv                : out STD_LOGIC;
           ld_p, sel_p           : out STD_LOGIC;
           ld_pabs   : out STD_LOGIC;
           sel_pabs  : out STD_LOGIC_VECTOR (1 downto 0);
```

```
            clr_signo, tog_signo   : out STD_LOGIC;
            clr_banco, ld_banco    : out STD_LOGIC;
            ld_tmp1, ld_tmp2       : out STD_LOGIC;
            ld_ma, sel_ma, ld_mb   : out STD_LOGIC;
            ld_t1, ld_t2, ld_t3    : out STD_LOGIC;
            ld_r, sel_r, clr_q, inc_q, ld_sq : out STD_LOGIC;
            ld_det    : out STD_LOGIC;
            sel_det   : out STD_LOGIC_VECTOR (1 downto 0);
            sel_sra   : out STD_LOGIC_VECTOR (1 downto 0);
            sel_srb   : out STD_LOGIC_VECTOR (2 downto 0);
            op_sr     : out STD_LOGIC;
            ld_selant, ld_neg      : out STD_LOGIC;
            ld_bin    : out STD_LOGIC;
            sel_bin   : out STD_LOGIC_VECTOR (1 downto 0);
            clr_bcd, ins_bcd, aj_bcd, sh_bcd : out STD_LOGIC;
            clr_cont, inc_cont     : out STD_LOGIC;
            clr_d, inc_d           : out STD_LOGIC;
            ld_disp                : out STD_LOGIC;
            final                  : out STD_LOGIC);
end component;

component camino_datos
    Port ( clk, rst  : in  STD_LOGIC;
            sw_mat    : in  STD_LOGIC_VECTOR (2 downto 0);
            sw_piv    : in  STD_LOGIC_VECTOR (2 downto 0);
            ld_pag, clr_dira, inc_dira : in STD_LOGIC;
            we_ram    : in  STD_LOGIC;
            sel_fila  : in  STD_LOGIC_VECTOR (1 downto 0);
            sel_col   : in  STD_LOGIC_VECTOR (1 downto 0);
            sel_din   : in  STD_LOGIC_VECTOR (2 downto 0);
            ld_n      : in  STD_LOGIC;
            ld_i      : in  STD_LOGIC;
            sel_i     : in  STD_LOGIC_VECTOR (1 downto 0);
            ld_j      : in  STD_LOGIC;
            sel_j     : in  STD_LOGIC_VECTOR (1 downto 0);
            ld_k, sel_k            : in STD_LOGIC;
            ld_piv                 : in STD_LOGIC;
            ld_p, sel_p            : in STD_LOGIC;
            ld_pabs   : in  STD_LOGIC;
            sel_pabs  : in  STD_LOGIC_VECTOR (1 downto 0);
            clr_signo, tog_signo   : in STD_LOGIC;
            clr_banco, ld_banco    : in STD_LOGIC;
            ld_tmp1, ld_tmp2       : in STD_LOGIC;
            ld_ma, sel_ma, ld_mb   : in STD_LOGIC;
            ld_t1, ld_t2, ld_t3    : in STD_LOGIC;
            ld_r, sel_r, clr_q, inc_q, ld_sq : in STD_LOGIC;
            ld_det    : in  STD_LOGIC;
            sel_det   : in  STD_LOGIC_VECTOR (1 downto 0);
            sel_sra   : in  STD_LOGIC_VECTOR (1 downto 0);
            sel_srb   : in  STD_LOGIC_VECTOR (2 downto 0);
            op_sr     : in  STD_LOGIC;
            ld_selant, ld_neg      : in STD_LOGIC;
            ld_bin    : in  STD_LOGIC;
            sel_bin   : in  STD_LOGIC_VECTOR (1 downto 0);
            clr_bcd, ins_bcd, aj_bcd, sh_bcd : in STD_LOGIC;
            clr_cont, inc_cont     : in STD_LOGIC;
            clr_d, inc_d           : in STD_LOGIC;
            ld_disp                : in STD_LOGIC;
            z_dout, i_eq_n, j_eq_n, k1_eq_n      : out STD_LOGIC;
            t3_neg, r_ge, sq, piv_neg            : out STD_LOGIC;
            signo, val_neg                       : out STD_LOGIC;
            cont_31, dig_gt4, d_9, sel_dif       : out STD_LOGIC;
            disp_bcd  : out STD_LOGIC_VECTOR (27 downto 0);
            disp_neg  : out STD_LOGIC;
            desborde  : out STD_LOGIC);
end component;

-- Senales de estado
signal z_dout, i_eq_n, j_eq_n, k1_eq_n : STD_LOGIC;
signal t3_neg, r_ge, sq, piv_neg       : STD_LOGIC;
signal signo, val_neg                  : STD_LOGIC;
signal cont_31, dig_gt4, d_9, sel_dif  : STD_LOGIC;

-- Senales de control
signal ld_pag, clr_dira, inc_dira : STD_LOGIC;
signal we_ram    : STD_LOGIC;
signal sel_fila  : STD_LOGIC_VECTOR (1 downto 0);
signal sel_col   : STD_LOGIC_VECTOR (1 downto 0);
signal sel_din   : STD_LOGIC_VECTOR (2 downto 0);
signal ld_n      : STD_LOGIC;
signal ld_i      : STD_LOGIC;
signal sel_i     : STD_LOGIC_VECTOR (1 downto 0);
signal ld_j      : STD_LOGIC;
signal sel_j     : STD_LOGIC_VECTOR (1 downto 0);
signal ld_k, sel_k            : STD_LOGIC;
signal ld_piv                 : STD_LOGIC;
signal ld_p, sel_p            : STD_LOGIC;
signal ld_pabs   : STD_LOGIC;
```

```
1230  signal sel_pabs  : STD_LOGIC_VECTOR (1 downto 0);
1231  signal clr_signo, tog_signo   : STD_LOGIC;
1232  signal clr_banco, ld_banco    : STD_LOGIC;
1233  signal ld_tmp1, ld_tmp2       : STD_LOGIC;
1234  signal ld_ma, sel_ma, ld_mb   : STD_LOGIC;
1235  signal ld_t1, ld_t2, ld_t3    : STD_LOGIC;
1236  signal ld_r, sel_r, clr_q, inc_q, ld_sq : STD_LOGIC;
1237  signal ld_det    : STD_LOGIC;
1238  signal sel_det   : STD_LOGIC_VECTOR (1 downto 0);
1239  signal sel_sra   : STD_LOGIC_VECTOR (1 downto 0);
1240  signal sel_srb   : STD_LOGIC_VECTOR (2 downto 0);
1241  signal op_sr     : STD_LOGIC;
1242  signal ld_selant, ld_neg      : STD_LOGIC;
1243  signal ld_bin    : STD_LOGIC;
1244  signal sel_bin   : STD_LOGIC_VECTOR (1 downto 0);
1245  signal clr_bcd, ins_bcd, aj_bcd, sh_bcd : STD_LOGIC;
1246  signal clr_cont, inc_cont     : STD_LOGIC;
1247  signal clr_d, inc_d           : STD_LOGIC;
1248  signal ld_disp                : STD_LOGIC;
1249
1250  begin
1251
1252  U_UC : unidad_control port map (
1253      clk => clk, rst => rst, inicio => inicio,
1254      z_dout => z_dout, i_eq_n => i_eq_n, j_eq_n => j_eq_n, k1_eq_n => k1_eq_n,
1255      t3_neg => t3_neg, r_ge => r_ge, sq => sq, piv_neg => piv_neg,
1256      signo => signo, val_neg => val_neg,
1257      cont_31 => cont_31, dig_gt4 => dig_gt4, d_9 => d_9, sel_dif => sel_dif,
1258      ld_pag => ld_pag, clr_dira => clr_dira, inc_dira => inc_dira,
1259      we_ram => we_ram, sel_fila => sel_fila, sel_col => sel_col, sel_din => sel_din,
1260      ld_n => ld_n, ld_i => ld_i, sel_i => sel_i, ld_j => ld_j, sel_j => sel_j,
1261      ld_k => ld_k, sel_k => sel_k, ld_piv => ld_piv, ld_p => ld_p, sel_p => sel_p,
1262      ld_pabs => ld_pabs, sel_pabs => sel_pabs,
1263      clr_signo => clr_signo, tog_signo => tog_signo,
1264      clr_banco => clr_banco, ld_banco => ld_banco,
1265      ld_tmp1 => ld_tmp1, ld_tmp2 => ld_tmp2,
1266      ld_ma => ld_ma, sel_ma => sel_ma, ld_mb => ld_mb,
1267      ld_t1 => ld_t1, ld_t2 => ld_t2, ld_t3 => ld_t3,
1268      ld_r => ld_r, sel_r => sel_r, clr_q => clr_q, inc_q => inc_q, ld_sq => ld_sq,
1269      ld_det => ld_det, sel_det => sel_det,
1270      sel_sra => sel_sra, sel_srb => sel_srb, op_sr => op_sr,
1271      ld_selant => ld_selant, ld_neg => ld_neg, ld_bin => ld_bin, sel_bin => sel_bin,
1272      clr_bcd => clr_bcd, ins_bcd => ins_bcd, aj_bcd => aj_bcd, sh_bcd => sh_bcd,
1273      clr_cont => clr_cont, inc_cont => inc_cont, clr_d => clr_d, inc_d => inc_d,
1274      ld_disp => ld_disp, final => final);
1275
1276  U_CD : camino_datos port map (
1277      clk => clk, rst => rst, sw_mat => sw_mat, sw_piv => sw_piv,
1278      ld_pag => ld_pag, clr_dira => clr_dira, inc_dira => inc_dira,
1279      we_ram => we_ram, sel_fila => sel_fila, sel_col => sel_col, sel_din => sel_din,
1280      ld_n => ld_n, ld_i => ld_i, sel_i => sel_i, ld_j => ld_j, sel_j => sel_j,
1281      ld_k => ld_k, sel_k => sel_k, ld_piv => ld_piv, ld_p => ld_p, sel_p => sel_p,
1282      ld_pabs => ld_pabs, sel_pabs => sel_pabs,
1283      clr_signo => clr_signo, tog_signo => tog_signo,
1284      clr_banco => clr_banco, ld_banco => ld_banco,
1285      ld_tmp1 => ld_tmp1, ld_tmp2 => ld_tmp2,
1286      ld_ma => ld_ma, sel_ma => sel_ma, ld_mb => ld_mb,
1287      ld_t1 => ld_t1, ld_t2 => ld_t2, ld_t3 => ld_t3,
1288      ld_r => ld_r, sel_r => sel_r, clr_q => clr_q, inc_q => inc_q, ld_sq => ld_sq,
1289      ld_det => ld_det, sel_det => sel_det,
1290      sel_sra => sel_sra, sel_srb => sel_srb, op_sr => op_sr,
1291      ld_selant => ld_selant, ld_neg => ld_neg, ld_bin => ld_bin, sel_bin => sel_bin,
1292      clr_bcd => clr_bcd, ins_bcd => ins_bcd, aj_bcd => aj_bcd, sh_bcd => sh_bcd,
1293      clr_cont => clr_cont, inc_cont => inc_cont, clr_d => clr_d, inc_d => inc_d,
1294      ld_disp => ld_disp,
1295      z_dout => z_dout, i_eq_n => i_eq_n, j_eq_n => j_eq_n, k1_eq_n => k1_eq_n,
1296      t3_neg => t3_neg, r_ge => r_ge, sq => sq, piv_neg => piv_neg,
1297      signo => signo, val_neg => val_neg,
1298      cont_31 => cont_31, dig_gt4 => dig_gt4, d_9 => d_9, sel_dif => sel_dif,
1299      disp_bcd => disp_bcd, disp_neg => disp_neg, desborde => desborde);
1300
1301  end Estructural;
1302
1303
1304  ----------------------------------------------------------------------------------
1305  -- SINCRONIZADOR: dos flip-flops para el boton y los switches
1306  ----------------------------------------------------------------------------------
1307  library IEEE;
1308  use IEEE.STD_LOGIC_1164.ALL;
1309
1310  entity sincronizador is
1311      Port ( clk, rst : in  STD_LOGIC;
1312             inicio   : in  STD_LOGIC;
1313             sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);
1314             sw_piv   : in  STD_LOGIC_VECTOR (2 downto 0);
1315             inicio_s : out STD_LOGIC;
1316             sw_mat_s : out STD_LOGIC_VECTOR (2 downto 0);
1317             sw_piv_s : out STD_LOGIC_VECTOR (2 downto 0));
```

```
end sincronizador;

architecture RTL of sincronizador is
signal ini_1, ini_2 : STD_LOGIC;
signal mat_1, mat_2, piv_1, piv_2 : STD_LOGIC_VECTOR (2 downto 0);
begin

process (clk, rst)
begin
    if rst = '1' then
        ini_1 <= '0';    ini_2 <= '0';
        mat_1 <= "000";  mat_2 <= "000";
        piv_1 <= "000";  piv_2 <= "000";
    elsif clk'event and clk = '1' then
        ini_1 <= inicio;  ini_2 <= ini_1;
        mat_1 <= sw_mat;  mat_2 <= mat_1;
        piv_1 <= sw_piv;  piv_2 <= piv_1;
    end if;
end process;

inicio_s <= ini_2;
sw_mat_s <= mat_2;
sw_piv_s <= piv_2;

end RTL;


----------------------------------------------------------------------------------
-- DISPLAY DE 8 DIGITOS MULTIPLEXADO (Nexys A7, anodo comun, activos en bajo)
-- AN7 = signo ('-' o apagado), AN6..AN0 = 7 digitos BCD.
-- Contador de refresco de 18 bits: 2^15 ciclos por digito, 2,6 ms por barrido.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity display_8dig is
    Port ( clk, rst : in  STD_LOGIC;
           bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
           neg      : in  STD_LOGIC;
           an       : out STD_LOGIC_VECTOR (7 downto 0);
           seg      : out STD_LOGIC_VECTOR (6 downto 0);   -- seg(0)=CA ... seg(6)=CG
           dp       : out STD_LOGIC);
end display_8dig;

architecture RTL of display_8dig is
signal refresco  : STD_LOGIC_VECTOR (17 downto 0);
signal dsel      : STD_LOGIC_VECTOR (2 downto 0);
signal car       : STD_LOGIC_VECTOR (3 downto 0);   -- 0-9, A = '-', F = apagado
signal car_signo : STD_LOGIC_VECTOR (3 downto 0);
begin

process (clk, rst)
begin
    if rst = '1' then
        refresco <= (others => '0');
    elsif clk'event and clk = '1' then
        refresco <= refresco + '1';
    end if;
end process;

dsel      <= refresco(17 downto 15);
car_signo <= "1010" when neg = '1' else "1111";

with dsel select
    car <= bcd(3 downto 0)   when "000",
           bcd(7 downto 4)   when "001",
           bcd(11 downto 8)  when "010",
           bcd(15 downto 12) when "011",
           bcd(19 downto 16) when "100",
           bcd(23 downto 20) when "101",
           bcd(27 downto 24) when "110",
           car_signo         when others;

with dsel select
    an <= "11111110" when "000",
          "11111101" when "001",
          "11111011" when "010",
          "11110111" when "011",
          "11101111" when "100",
          "11011111" when "101",
          "10111111" when "110",
          "01111111" when others;

with car select                                 --  gfedcba
    seg <= "1000000" when "0000",               --  0
           "1111001" when "0001",               --  1
           "0100100" when "0010",               --  2
```

```
            "0110000" when "0011",              --  3
            "0011001" when "0100",              --  4
            "0010010" when "0101",              --  5
            "0000010" when "0110",              --  6
            "1111000" when "0111",              --  7
            "0000000" when "1000",              --  8
            "0010000" when "1001",              --  9
            "0111111" when "1010",              --  -
            "1111111" when others;              --  apagado

dp <= '1';

end RTL;


--------------------------------------------------------------------------------
-- TOPE: det_bareiss (descripcion estructural)
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity det_bareiss is
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;                      -- BTNC
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW15..SW13: matriz (pagina de la ROM)
           sw_piv   : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW2..SW0 : 0 = det, 1..4 = pivote k
           final    : out STD_LOGIC;                      -- LD0
           desborde : out STD_LOGIC;                      -- LD1: |valor| > 9 999 999
           an       : out STD_LOGIC_VECTOR (7 downto 0);
           seg      : out STD_LOGIC_VECTOR (6 downto 0);
           dp       : out STD_LOGIC);
end det_bareiss;

architecture Estructural of det_bareiss is

component sincronizador
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);
           sw_piv   : in  STD_LOGIC_VECTOR (2 downto 0);
           inicio_s : out STD_LOGIC;
           sw_mat_s : out STD_LOGIC_VECTOR (2 downto 0);
           sw_piv_s : out STD_LOGIC_VECTOR (2 downto 0));
end component;

component procesador
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);
           sw_piv   : in  STD_LOGIC_VECTOR (2 downto 0);
           final    : out STD_LOGIC;
           disp_bcd : out STD_LOGIC_VECTOR (27 downto 0);
           disp_neg : out STD_LOGIC;
           desborde : out STD_LOGIC);
end component;

component display_8dig
    Port ( clk, rst : in  STD_LOGIC;
           bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
           neg      : in  STD_LOGIC;
           an       : out STD_LOGIC_VECTOR (7 downto 0);
           seg      : out STD_LOGIC_VECTOR (6 downto 0);
           dp       : out STD_LOGIC);
end component;

signal inicio_s : STD_LOGIC;
signal sw_mat_s, sw_piv_s : STD_LOGIC_VECTOR (2 downto 0);
signal disp_bcd : STD_LOGIC_VECTOR (27 downto 0);
signal disp_neg : STD_LOGIC;

begin

U_SINC : sincronizador port map (
    clk => clk, rst => rst, inicio => inicio, sw_mat => sw_mat, sw_piv => sw_piv,
    inicio_s => inicio_s, sw_mat_s => sw_mat_s, sw_piv_s => sw_piv_s);

U_PROC : procesador port map (
    clk => clk, rst => rst, inicio => inicio_s, sw_mat => sw_mat_s, sw_piv => sw_piv_s,
    final => final, disp_bcd => disp_bcd, disp_neg => disp_neg, desborde => desborde);

U_DISP : display_8dig port map (
    clk => clk, rst => rst, bcd => disp_bcd, neg => disp_neg,
    an => an, seg => seg, dp => dp);

end Estructural;
```